

FOOTBALL MATCH PREDICTION USING MACHINE LEARNING

**by
Elif Dikmen**

Engineering Project Report

**Yeditepe University
Faculty of Engineering
Department of Computer Engineering
2025**

FOOTBALL MATCH PREDICTION USING MACHINE LEARNING

APPROVED BY:

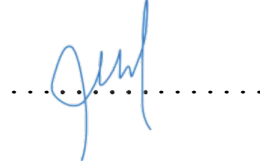
Prof. Dr. Şebnem Baydere
(Supervisor)



Assist. Prof. Dr. Esin Onbaşlıoğlu



Lecturer Betül Arslan



DATE OF APPROVAL: .../.../2025

ACKNOWLEDGEMENTS

First of all I would like to thank my advisor Prof. Dr. Şebnem Baydere for her guidance and support throughout my project.

Also I would like to thank my parents for their support and encouragement throughout my education up to the present.

ABSTRACT

FOOTBALL MATCH PREDICTION USING MACHINE LEARNING

This paper discusses a study on the potential for machine learning to be applied in the scope of future and live football match outcomes prediction based on publicly available data from five major European leagues, Turkish Super League, FIFA Club World Cup and World Cup Qualification Europe. Leverage the increasing availability of structured sports data and evaluate three popular classification models: XGBoost, Random Forest and Logistic Regression. For the analysis of predictive performance, two experimental scenarios were designed. For the experimental setup, match data from before 2023 was taken for training the models. All matches played in 2023 and onwards were considered unseen data and were used for testing. The models were analyzed with classification metrics: precision, recall, and F1-score, accuracy. The results show that Random Forest outperformed in future and live matches but Logistic Regression was better in two leagues for future matches. Overall, Random Forest gave better results in most cases.

ÖZET

TITLE OF THE PROJECT

Bu makale, makine öğrenmesi tekniklerinin gelecek ve canlı futbol maç sonuçlarını tahmin etme amacıyla kullanım potansiyelini inceleyen bir çalışmayı ele almaktadır. Çalışmada, beş büyük Avrupa ligi, Türkiye Süper Ligi, FIFA Kulüpler Dünya Kupası ve Avrupa Dünya Kupası Elemeleri'nden elde edilen kamuya açık veriler kullanılmıştır. Yapılandırılmış spor verilerinin giderek artan erişilebilirliğinden yararlanılarak üç popüler sınıflandırma modeli XGBoost, Random Forest ve Lojistik Regresyon değerlendirilmiştir. Tahmin performansını analiz edebilmek için iki deneysel senaryo tasarlanmıştır. İlk senaryoda, 2023 ve öncesinde oynanan maçlar modelin eğitimi için kullanılmıştır. İkinci senaryoda ise 2023 yılı ve sonrasında oynanan tüm maçlar ise görülmemiş veri olarak değerlendirilmiş ve test amacıyla kullanılmıştır.. Modeller; doğruluk , kesinlik, duyarlılık ve F1-skoru gibi sınıflandırma metrikleriyle analiz edilmiştir. Sonuçlar, hem gelecek hem canlı maçlar için Random Forest modelinin genel olarak daha iyi performans gösterdiğini, ancak Lojistik Regresyon'un gelecek maçlarda iki ligde daha başarılı sonuçlar verdiğini ortaya koymuştur. Genel olarak, Random Forest çoğu durumda daha iyi sonuçlar üretmiştir.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
ÖZET	v
LIST OF FIGURES	viii
LIST OF TABLES	x
LIST OF SYMBOLS/ABBREVIATIONS	xi
1. INTRODUCTION	1
1.1. Problem Definition	2
1.2. Motivation	2
1.3. Requirements	3
2. RELATED WORK	5
2.1. Machine Learning Approaches for Football Match Prediction	5
2.2. Real-Time Data and Dynamic Prediction Models	6
2.3. Explainability and Interpretability in Sports Prediction	6
2.4. Gaps in Interpretability and Real-World Deployment	7
2.5. Summary and Implications for This Study	8
3. ANALYSIS & DESIGN	9
3.1. Functional Requirements	9
3.2. Non Functional Requirements	10
3.3. System Architecture	11
3.4. Diagrams	12
3.4.1. Use Case Diagram for Match Prediction App	12
3.4.2. Sequence Diagram for Viewing Match Details	13
3.4.3. Sequence Diagram Predicting Football Matches	15
3.4.4. Database E-R Diagram	16
4. IMPLEMENTATION	18
4.1. Data Collection	18
4.2. Data Preprocessing	19
4.2.1. Missing Value Treatment and Data Cleaning	19
4.2.2. Feature Type Normalization and Object to Numeric Conversion	19
4.2.3. Feature Engineering During Preprocessing	20
4.2.4. Feature Selection	22
4.2.5. Live Match Model and Real-Time Feature Integration	24
4.2.6. Standardization and Merging of League Datasets	25
4.2.7. Final Dataset Structure	26

4.3. Machine Learning Algorithms	26
4.3.1. Random Forest	26
4.3.2. Extreme Gradient Boosting	27
4.3.3. Logistic Regression	29
4.4. Model Selection	30
4.5. IOS Application	31
4.5.1. Setup and Initialization	31
4.5.2. User Interface	31
4.5.3. Model Integration and Prediction Process	35
4.5.4. Design and User Experience	35
5. TEST AND RESULTS	36
5.1. Evaluation on Future Match Predictions	36
5.1.1. Classification Metrics Overview	37
5.2. Evaluation on Live Matches	38
5.3. Model Training and Validation Results	39
5.4. System Latency Evaluation	43
6. CONCLUSION	45
6.1. Future Work	45
Bibliography	46
APPENDIX A: CONFUSION MATRICES	48

LIST OF FIGURES

3.1	The overall system architecture.	12
3.2	Use case diagram of user interactions with the application.	13
3.3	Sequence Diagram for Detail Page.	14
3.4	Sequence Diagram for Match Prediction	16
3.5	Entity-Relationship Diagram of the Database Schema	17
4.1	Feature importance	23
4.2	Live Matches Feature importance	25
4.3	Random Forest Algorithm illustration [16]	27
4.4	Visualization of logistic regression.	29
4.5	Home Page/Finished Matches	32
4.6	Filter Leagues	32
4.7	Detail Page/Standings	33
4.8	Detail Page/Events	33
4.9	Home Page/Future Matches	34
4.10	Live Matches	34
5.1	Confusion Matrix for all leagues	37
5.2	Confusion Matrix for Live matches	39
5.3	Latency Distribution	44
A.1	Bundesliga – Logistic Regression	48
A.2	Bundesliga – XGBoost	48
A.3	Bundesliga – Random Forest	48
A.4	Turkish Super League – Random Forest	48
A.5	Turkish Super League – XGBoost	49
A.6	Turkish Super League – Logistic Regression	49
A.7	EPL – Logistic Regression	49
A.8	EPL – XGBoost	49
A.9	EPL – Random Forest	49
A.10	La Liga – Logistic Regression	49
A.11	La Liga – XGBoost	50
A.12	La Liga – Random Forest	50
A.13	Serie A – Logistic Regression	50
A.14	Serie A – XGBoost	50
A.15	Serie A – Random Forest	50
A.16	French League 1 – Logistic Regression	50
A.17	French League 1 – XGBoost	51

A.18	French League 1 – Random Forest	51
A.19	FIFA Club World Cup – Logistic Regression	51
A.20	FIFA Club World Cup – XGBoost	51
A.21	FIFA Club World Cup – Random Forest	51
A.22	World Cup Europe – Logistic Regression	51
A.23	World Cup Europe – XGBoost	52
A.24	World Cup Europe–Random Forest	52

LIST OF TABLES

4.1	List of engineered and extracted features used in the model.	24
4.2	List of engineered and extracted features used in the model.	25
5.1	Classification performance metrics of the model.	38
5.2	Classification performance metrics of the live matches model.	39
5.3	Bundesliga train test model results	40
5.4	English Premier League train test model results	41
5.5	Turkish Super Lig train test model results	41
5.6	La Liga train test model results	42
5.7	Serie A train test model results	42
5.8	French 1 train test results	42
5.9	FIFA Club World Cup train test model results	43
5.10	World Cup Qualification Europe train test model results	43

LIST OF SYMBOLS/ABBREVIATIONS

e.g	for example
i.e	in other words
ML	Machine Learning
HomeTeam_code	Unique code of home team
AwayTeam_code	Unique code of away team
FTR	Full time result
league	Name of league
E0	English Premier League
T1	Turkish Super League
I1	Italy League
Serie A	Italy League
French 1	French League
SP1	Spain League
La Liga	Spain League
D1	Germany League
Bundesliga	Germany League
ANN	Artificial Neural Network
RNN	Recurrent Neural Network
LSTM	Long short-term memory
LSTM	Long short-term memory
XAI	Explainable artificial intelligence
SHAP	(Shapley Additive Explanation
AxG	Away expected Goals
HxG	Home expected Goals
B365H	Bet365 odds for Home Win
B365A	Bet365 odds for Away Win
B365D	Bet365 odds for Draw
H2H	Head to Head
H	Home Win
A	Away Win
D	Draw Win
WinRateDiff	Difference in win rates between the home team and the away team in last 5 matches
DrawRateDiff	Difference in draw rates between the home team and the away team in last 5 matches
Last5_WinRate_Home	Home team's win rate over their last 5 matches.

Last5_DrawRate_Home	Home team's draw rate over their last 5 matches.
Last5_LossRate_Home	Home team's loss rate over their last 5 matches.
Last5_WinRate_Away	Away team's win rate over their last 5 matches.
Last5_DrawRate_Away	Away team's draw rate over their last 5 matches.
Last5_LossRate_Away	Away team's loss rate over their last 5 matches.
Last5_Goals_Home	Average number of goals by the home team in last 5 matches.
Last5_Goals_Away	Average number of goals by the away team in last 5 matches.
HomeProb	Normalizing B365 odds to probabilities for the home team
AwayProb	Normalizing B365 odds to probabilities for the away team
DrawProb	Normalization of B365 odds to draw probability
EloDiff	Difference in Elo ratings between the home and the away team.
EloChange30_Home	Home team's change of Elo rating over the last 30 days
EloChange60_Home	Home team's change of Elo rating over the last 60 days
EloChange30_Away	Away team's change of Elo rating over the last 30 days
EloChange60_Away	Away team's change of Elo rating over the last 60 days
xG_diff	Difference of expected goals between home and away team

1. INTRODUCTION

Data-driven technologies have affected sports in recent years, and football has been affected the most in data collection, analysis, and utilization processes. Over the past few years, data-driven technologies have made strong inroads into the field of sports. Football has undergone a major change in the way data is collected, analyzed, and used. Currently, clubs, analysts, and researchers tend to heavily use large volumes of structured data, such as those relating to team performance parameters, player statistics, and historical records of matches, for decision-making, strategy formulation, player appraisal, and outcome prediction. With this shift has come the burgeoning discipline of sports analytics, in which ML has found its strength to analyze the complexities of the game. Nevertheless, making a prediction about who wins a particular football game is still very hard. Yet, football is a sport full of surprises—an unexpected injury, an unfair booking, a spillage of rain on the pitch, or maybe a goal in the dying seconds of the match can alter the whole course of the game. These uncertainties add a zip of fun to the game and make it nearly impossible to develop a foolproof prediction system. Nevertheless, patterns in historical data have shown tremendous potential in assisting human intelligence in fine-tuning systems for making more accurate predictions.

This thesis presents a football match prediction system based on machine learning that can produce results with accuracy and explainability. The platform is engineered for macOS and iOS, integrating a machine learning backend powered by Python with an Xcode-developed Swift frontend. The system predicts pre-match and in-play, depending on the availability of live data streams, to allow an analyst and forecaster to apply the method in real time. In order to select the best predictive modeling approach, various machine-learning algorithms were tried and compared. Each method was subjected to well-known validation approaches and integrated into the final application the best-performing method. Besides prediction accuracy, the interpretability question is of high importance within the work. One of the major pitfalls of most existing systems is that there is no explanation for the predictions they provide. To counter this, the proposed application introduces a set of tools to guide the user with an understanding of what exactly influenced each prediction, including measures like team form, possession percentages or recent results. This mechanism, hence, nurtures end-user trust and further extends the system's utility to fans, analysts, and coaching staff. The practical deployment aspect attempts to bridge the gap between academic research and real-world platform-specific mobile applications through machine learning. The remainder of this chapter, therefore, will describe the actual problem the project has set out to address and the technical and functional requirements that must be met for the successful implementation of the solution.

1.1. Problem Definition

Prediction of soccer match results might be a complicated one; however, it is intensely researched from the fields of machine learning, statistical modeling, and sports analytics. Football is a low-scoring and highly dynamic sport, with match outcomes influenced by structured data and unstructured random ones, like weather conditions, a referee's decision, or a sudden surge in player motivation. This interaction between the measurable and random introduces a layer of uncertainty, which limits the capability of deterministic models. Huge advances in data acquisition and machine learning have been made in recent times; however, one of the most difficult things to do is to have models with high predictive accuracy. At times, a single instance can change the entire scenario of a match for example a goal in the dying minutes or a player getting injured, only the fine nuances being easy to get lost in static data. Another common shortcoming of pre-existing systems is that they usually emphasize pre-match prediction rather than dynamic shifts during the course of play. Applications of the prediction are subject to dynamic instances during play, so for any practical prediction solution, live updates should be employed in forecast adjustment. In the view of several other factors, many classical models lack interpretability as well. Complex algorithms may reach a certain degree of accuracy, yet they prove to be black boxes that give no indication of what led to a certain prediction. This stands in the way of their real-world adoption, mostly for end-users such as coaches, analysts, or even the fans who need transparent and explainable outcomes to base decisions on. Another issue is that the majority of prior work has been confined to experimental and academic scenarios, failing to translate these predictive models into accessible, deployable applications. It, thus, weaves together some classical machine learning methods, live data integration, and explanatory feedback into a working application for macOS and iOS. With it, the system hopes to fill the gap and join hands with sports analytics in taking predictive tools from the realm of academic research to practical, user-facing applications.

1.2. Motivation

Employment of data-driven techniques in sports has revolutionized the study of games, team management, and player analysis. Football being a popularly viewed and commercially most lucrative sport produces piles of structured and semi-structured data that, if used properly, has great predictive capabilities. Despite advancements in machine learning, much of the research conducted earlier on football match prediction has been in the nature of theoretical research or experiments, with minimal implementation in actual end-user tools.

One of the main reasons for this work is an imbalance between mathematical machine

learning models and practical, deployable applications that are interpretable, understandable, and responsive. The majority of present systems react in a black-box manner and provide minimal to no explanation as to why a particular result is expected. Such openness limits user trust and real-world usability, especially among constituents such as coaches, sports commentators, and even fans who not only require accurate predictions but also the reasoning behind them. Besides, most predictive models are static in nature and only use pre-match information and fail to capture real-time changes in action during a game. Live match updates can have tremendous impact on relevance and accuracy of prediction, but very few systems have ventured in this because of technical complexity.

The widespread adoption of mobile and desktop platforms also presents a possibility to further engage smart systems among users. Well-designed, native application with the capacity to provide accurate and interpretable football predictions could be not only of scholarly interest but also of pragmatic concern in sports strategy, fan engagement, and even in moral betting models. This project is thus inspired by the need to close the gap between usability and research through creating a machine learning-based football match prediction system that is not only accurate but also transparent, real-time responsive, and platform-optimized for macOS and iOS devices. By means of this project, it is hoped to contribute to the newly forming discipline of applied sports analytics and show how advanced machine learning models can be translated into useful, interactive tools for real-world decision-making.

1.3. Requirements

The prediction app for football matches is built on a modular hardware and software platform that supports both model development and deployment on macOS and iOS operating systems. It aims to deliver precise, interpretable predictions with minimal latency and acceptable usability. The system was installed on a MacBook Pro Apple M1 chip, which provides native support for efficient on-device computation and integration within the Apple ecosystem. The hardware platform offers fast execution of data processing and model training tasks without depending on external services. The software stack is centered around Python, which was used for all of the data collection, preprocessing, model training, and evaluation tasks. Key libraries that were used included Pandas and NumPy for ordered data manipulation, Matplotlib and Seaborn for data visualization, and Scikit-learn for specifying the machine learning models. XGBoost was also used for the assessment of gradient boosting models. The user interface of the application was developed using Swift in Xcode, with native support for macOS and iOS. The application gathers some football match datas from football APIs, which provide information such as fixtures, standings and events . Where features were not immediately available via API endpoints, web scraping techniques were implemented to

retrieve them, such as momentum indicators, form trends, or player ratings from sport media websites. All of the data gathered is stored in a SQLite database, which was selected because it is lightweight, has fast access, and is easily integrated into both the Python and mobile environments. This database serves as the central repository for model input and user-displayable data. To build a strong predictive engine, various machine learning models were developed and compared, including Logistic Regression, Random Forest, and XGBoost. The models were evaluated by k-fold cross-validation and optimized through grid search hyperparameter tuning. Model comparison was done on performance metrics such as accuracy, precision, recall, and F1-score. Random Forest and Logistic Regression were selected as the best models from the results obtained because they had stronger predictive capability and interpretability than XGBoost. The prediction method is framed as a multiclass classification problem, and the output is one of three options: home win, draw, or away win. The chosen model is embedded in the app and provides predictions along with explanatory information. For instance, users can view the most significant features for an individual match prediction, increasing trust and understanding. Classification metrics like accuracy, precision, recall, and F1-score are used to compare performance across models between algorithms. While neither formal user testing nor application-level performance measures were part of this research, these remain intriguing directions for future work. The current implementation focuses on reliable model development, lightweight storage through SQLite, real-time data integration, and native Apple UI development through Swift and Xcode. Together, these components aim to deliver a usable and technically proficient football game prediction tool with a focus on both accuracy and interpretability.

2. RELATED WORK

2.1. Machine Learning Approaches for Football Match Prediction

Machine learning is now a foundation of sports analytics, providing rich opportunities for pattern recognition, classification, and prediction. In football match outcome prediction, machine learning algorithms are ubiquitous in analyzing past data and identifying patterns that can predict match outcomes in the future. These are typically framed as multiclass classification problems, where the model is to predict one of the three: home win, draw, or away win. A large number of different machine learning algorithms have been employed in previous work to address this issue. The most popular used are Logistic Regression, Random Forest, Support Vector Machine, Gradient Boosting, and more recently, Deep Learning architectures such as Recurrent Neural Networks and Long Short-Term Memory networks. They have been tried on various datasets ranging from single-season league data to cumulative multi-season statistics. Prasetio and Harlili (2016) used a Logistic Regression model as input features using FIFA game ratings and achieved 69.5 prediction accuracy when trained on five previous English Premier League seasons [1]. However, their model failed to predict the draw outcome, so it is not completely usable in real-world implementations. Similarly, Igiri and Nwachukwu (2017) compared Logistic Regression and Artificial Neural Networks against in-game statistics such as shots on target and yellow cards. Their single-season EPL model demonstrated an extremely high accuracy of %95, though the limited dataset along with post-match data made it improbable for the purpose of predicting future matches[2].

Snyder (2018) augmented the logistic model with extra contextual factors, such as stadium capacity and travel distance, for predicting post-match outcomes. Although innovative in feature selection, the model was only moderately accurate, highlighting the difficulty of the task with sparse pre-match data[3]. Pugsee and Pattawong (2019) tested Random Forest and Multilayer Perceptron models on three consecutive EPL seasons. They demonstrated that Random Forest performs better than MLP in any case of binary classification such as "home win vs. not home win", and the best accuracy discovered was %81.81[4] in their experiment. In a comparative study by Alfredo and Isa (2020), tree-based models like Random Forest, C5.0, and XGBoost were validated on a 10-season EPL dataset with 14 independent features. XGBoost and Random Forest displayed comparable performance with accuracies greater than %67, confirming the robustness of ensemble methods in sports forecasting issues[5].

2.2. Real-Time Data and Dynamic Prediction Models

Perhaps the most glaring advancement in sports analytics is incorporating real-time data into predictive models. Traditional predictive models for football match results have been focused on static pre match information such as historical team form, player ratings, and head-to-head stats. While these characteristics provide us with a basic idea of team strengths, they fail to capture the dynamic nature of football matches. More recent studies have begun to incorporate live data, such as the most recent match scores, red cards, possession percentages, and even in-play betting prices, to help improve predictive accuracy as the match unfolds. For example, Atta Mills et al. (2024) proposed a model that updates its predictions at halftime by incorporating features such as the most recent score and goal differences. Their results showed a spectacular improvement in forecasting accuracy[6] when live features were added, highlighting the significance of dynamic input. This thesis follows a similar approach by adding halftime data to the model so that predictions may be refreshed in real-time. This adaptability not only enhances model performance but also enables new applications such as dynamic coaching, real-time betting tips, and live strategy evaluation. Inheriting real-time data, though, introduces technical challenges: low-latency data pipelines, pre processing, and coping with the noise of live statistics. Satisfying these requirements is the key to releasing usable and responsive systems.

2.3. Explainability and Interpretability in Sports Prediction

As prediction systems become more complex with ensemble methods and deep learning architectures, the need for transparency and interpretability has increased. Explainable Artificial Intelligence techniques are used to ensure that predictions produced by models are trustworthy and understandable to domain experts, coaches, and end users. One of the streams of research relies on using inherently interpretable models or promoting black-box models with post-hoc explanation techniques. Yeung et al. (2021) suggested an interpretable model derived from XGBoost, putting special focus on feature importance scores for determining the variables having the most significant influence on the predicted outcomes [7]. Similarly, Plakias et al. (2022) utilized SHAP to provide explanations for model predictions for French top-tier league football match outcomes in Ligue 1. Their research revealed insights to act upon, such as short-passing approach's positive correlation with team achievement[8]. This thesis is a continuation of these trends by adding SHAP values to characterize match prediction at an individual level. The system not only makes predictions but also indicates the most important features contributing to each prediction. Focusing on interpretability provides additional reliability and usability, especially for non-expert users of the system. Collectively, these improvements demonstrate the increased use of bridging the gap between predictive

performance and actual usage. Incorporating real-time flexibility and interpretability into this perspective is what the thesis aims to add to a more holistic and user-friendly path in the field of football match prediction.

2.4. Gaps in Interpretability and Real-World Deployment

Despite remarkable progress in supervised learning-based football match prediction and advanced feature engineering, there are certain central gaps in the current literature—particularly around interpretability and usability in operational systems. Research is primarily focused on improving predictive accuracy based on pre-match static data but is not interested in creating dynamic, user-usable, and interpretable systems that operate in live environments. One critical drawback of most past research is opacity of interpretation. Although methods such as Random Forest or XGBoost have high predictive performance, they are typically black boxes with a lot of explaining not occurring within them, giving very little insight into the decision-making process itself[9]. This makes them less useful in practice, especially when domain experts require an explicit analysis of the key features underlying each prediction. Although post-hoc explanation methods such as SHAP were applied only in recent times within sports forecasting, their utilization for interactive applications is unusual[10]. End-users tend to receive output labels alone without further decomposition, lowering trust and usability. Aside from transparency concerns, most prediction systems remain offline-bound. The models are typically developed and validated in research environments without reaching production platforms, thereby ignoring real-world considerations such as model serving, latency, UI integration, and real-time data processing. As stressed by more recent work, incorporation of real-time match data, half-time scores or red card events, significantly boosts predictive accuracy[6]. Nevertheless, very few deployments have taken this route, largely due to the technical issues of dealing with real-time input streams, low-latency inference, and live feature engineering. There is a second gap at the device level, particularly in mobile or cross-device usage. While web-based dashboards are sometimes used to show model output, most platforms are not designed for macOS or iOS environments where on-device responsiveness, storage on device, and natively supported UI inputs are important. Moreover, very few prediction systems are user experience-oriented—discounting features such as simple visualization, user control, and user-customized feedback. Finally, assessment strategies in previous researchs have a tendency to be focused on technical metrics with little or no user-level validation. There is less evidence about the way that users perceive the utility, comprehensibility, or trustworthiness of these forecasts. Without qualitative feedback from real-world users, it is difficult to measure the actual impact or benefit of these systems outside of controlled trials. This thesis fills these gaps by proposing a system that incorporates robust machine learning with model explainability, real-time data fusion, and natively mobile

deployment. The result is a platform that not only makes accurate match predictions but also openly and understandably documents the reasons why, tailored to end-users

2.5. Summary and Implications for This Study

Current literature presents a significant improvement in the research of football match result prediction, particularly through the application of supervised machine learning algorithms.[5]. Recent progress has sought deep learning approaches such as RNNs and LSTMs[11] in order to preserve temporal trends and describe the dynamics of matches. Such approaches, exciting as they are, are still with significant limitations regarding predictive performance and practical implementation. One of the strongest issues highlighted in recent studies is that it's not possible to forecast match results accurately due to the randomness of football. Even the most advanced models usually only achieve 50–70% accuracy, performing better on majority classes and worse on draws or upsets[12]. Therefore, research has moved towards employing a larger suite of evaluation metrics—such as precision, recall, F1-score, ROC-AUC, and Brier scores—to achieve a more granular perspective on model behavior for all classes of outcomes. A new theme is the use of live data. Models have relied on pre match attributes, but recent research stresses the need to use live match statistics[6]. This includes half time scores and goal events. Such data dynamically updates predictions as a game plays out. This can enable more accurate of great value for in-play decision-making or gambling. The integration of real time data introduces new engineering and latency limits. For instance, data pipelines must remain stable, plus predictions in streaming environments need to be dependable. People increasingly see explainable artificial intelligence (XAI) as a requirement for sports forecasting. Ensemble and deep neural networks, while powerful, are often relegated to being treated as black boxes. In order to deliver confidence of trust and adoption by analysts, coaches, and end-users, there is a call in the recent literature to utilize XAI tools like Shapley Additive Explanations[9] and feature importance rankings to explain how a model comes to a given prediction. They help bridge the predictability-performance gap and actionable understanding. Most models remain confined to research settings without being translated into tools available for use by fans, gamblers, or even career analysts. Mobile deployment may offer a new option for bringing predictions and explanations to consumers in real time, but this potential remains under-explored in the existing literature. This thesis attempts to address these gaps by combining accurate match result prediction with real-time data integration and real-world implementation into a mobile app. It thereby progresses both intellectual debate and real-world utility in applying predictive analysis to football. The approach not only capitalizes on known practices but also extends them by showing how machine learning models can act as responsive, and user-centric decision-support systems within the dynamic live sports.

3. ANALYSIS & DESIGN

3.1. Functional Requirements

Functional requirements outline functions and behaviors the system must have in order for it to be capable to execute its function. For this project — a system for predicting football matches using machine learning with real-time data capabilities as well as a user interface — functional requirements include:

(i) **Predicting historical matches**

The system ought to be capable of displaying predicted match results (Home Win, Draw, Away Win) for future matches using past data. The system ought to utilize pre-trained machine learning models in making such predictions. Predictions must be refreshed periodically as the dataset grows with new seasons.

(ii) **Live Match Prediction**

The system has to dynamically update predictions in real-time, even within live games. The system should have in-game information such as half-time score, cards, and goals for improving mid-game prediction accuracy.

(iii) **Probability Output**

The system should be producing outcome probabilities for all single matches.

(iv) **Prediction Explanation**

The system needs to give interpretability results indicating what features assisted in making a prediction. The system must include feature importance explainability components.

(v) **League and Team Filtering**

The system must permit filtering of predictions on a particular league.

(vi) **Data Integration**

The system must merge data from static datasets with dynamic APIs . The system must solve team naming conflicts using team naming mapping logic.

(vii) **Prediction Interface**

The system should have an easy-to-use graphical interface to display predictions, explanations, and updates in real-time. The interface should be optimized for macOS as well as iOS.

(viii) **Model Selection and Performance**

The system should compare and train various models and choose the best-performing model on a given set of provided metrics. It must include accuracy, precision, recall, and F1-score for all the models.

(ix) **Real Time API Connectivity**

The system must be integrated with real-time match schedule, odds, and in-game event data APIs. The API calls must be processed asynchronously in order to update in real-time without compromising your UI responsiveness.

(x) **Mapping and Storing Data**

The system should maintain proper match information in light storage. The system ought to store API responses as well as preprocessed information for efficient reuse in making predictions.

3.2. Non Functional Requirements

Non-functional requirements identify quality characteristics and performance limitations of the system. They ensure not only that the system performs its intended tasks but in an efficient, dependable, secure, and usable manner. These are the non-functional requirements of the football prediction system as given below:

(i) **Performance**

The system should provide prediction results within 2 seconds as soon as a user selects a match. Prediction updates must be processed and displayed on the user interface almost in real time (within ≤ 5 seconds latency) in live match scenarios. SQLite database should return read/write queries in under 100 milliseconds under normal conditions.

(ii) **Scalability**

It must be scalable in order to incorporate more seasons or leagues with minimal architectural modifications. Columns of new features or additional team statistics should be easily integrated into the pipeline with minimal modifications.

(iii) **Availability and Reliability**

The application should be offline-enabled for prediction before a match, using data stored locally. In case an API is not accessible or no live data is present, it should gracefully fallback to using pre-match data only.

(iv) **Maintainability**

The system should have a well-documented codebase that is modular, making it easily

extensible or modifiable in its features or in model integration for future developers. Thresholds for model updates and update intervals are configuration parameters and ought to be externalized using config files.

(v) **Portability**

The application should be Apple platform compatible (macOS, iOS) with consistent UI/UX. It should support both Intel and Apple Silicon (M1/M2) architectures without performance degradation.

(vi) **User**

It should have a customer-oriented interface with minimal customer training or instruction required. All users ought to be able to get a prediction, an explanation, and matching details within three interactions. Predictions should be presented with prominent visual cues in order to aid interpretability.

3.3. System Architecture

The architecture of the proposed football match prediction system is a pipeline modular architecture with a sequential structure, as illustrated in Figure 3.1. It consists of a number of steps including data collection, preprocessing, feature engineering, training, and prediction. The following diagram gives an overview of the architecture. It begins with Web Scraping, collecting structured information such as xG, recent form stats, and odds from other websites. External information is then enhanced with historical match history collected via APIs as well as open datasets, including rich contextual information for prediction models. Data, once collected, is loaded into the data collection module. Here, data from different leagues and from different sources is merged into a uniform storage system. Consistency checks, as well as schema unification, are applied in order to standardise column structures between different leagues. Then, there is the Data Preprocessing phase which involves imputation of missing data, categorical variable transformation, date normalization, and team name encoding. Team name mapping is also done for consistency with historic datasets as well as live fixture APIs. The feature engineering module then adds statistically computed features in order to enrich the dataset. These include rolling averages of goals, win/draw/loss ratios, time-windowed ELO rating differentials, as well as normalization of bookmaker odds expressed in probability space. These types of features are crucial in capturing match performances. Preprocessed data is input into the ML Algorithm module. A comparison between various machine learning classifiers like Random Forest, Logistic Regression is made on past data in training. Best model is optimized with cross-validation, as well as hyperparameter tuning, and Trained Model is stored for future use. Finally, Prediction creates outcome predictions from new game data using the learned model. Predictions are provided in pre-match

as well as live mode depending on input availability. Overall, the architecture is following a generic machine learning pipeline such that the system remains scalable, maintainable, and extensible to any potential future enhancements such as deeper learning architecture or other sources of data.

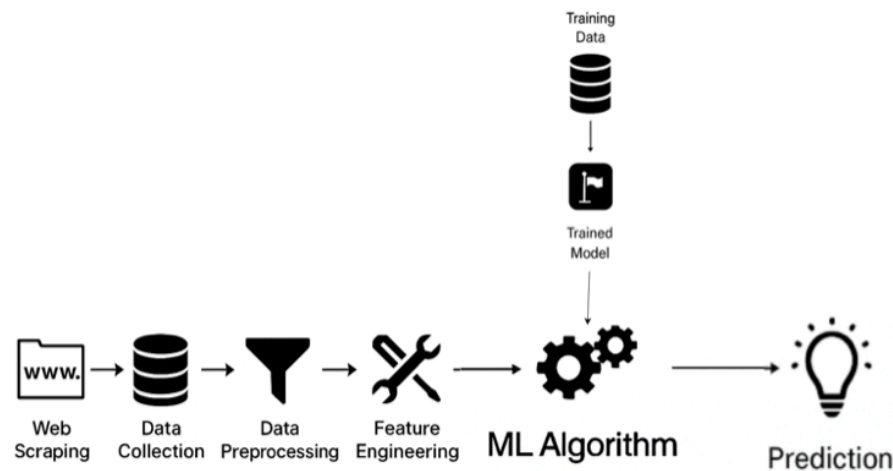


Figure 3.1: The overall system architecture.

3.4. Diagrams

3.4.1. Use Case Diagram for Match Prediction App

This use case diagram defines the main interactions between the user and the football match prediction system, as shown in Figure 3.2. The primary actor is the User, and the actor may interact with the system via several functionalities concerning match fixtures and predictions. The View Match Fixtures use case enables users to browse upcoming matches. This action includes fetching pre-match predictions and can be extended to viewing more information via the 'Check Out Why' Button or 'View Match Details'. View Match Details is a central function that provides deeper insight into individual matches. It includes three sub-use cases: View Match Statistics, which displays historical or recent match data. View Events: Shows timeline-based match events. View Standings: Presents league table standings for the involved teams. On the live match side, View Live Matches allows users to see currently ongoing games. This use case includes retrieving Live Predictions, which are dynamically updated using real-time data. Optionally, the View Match Details flow can also be extended to live matches to inspect evolving stats. Users also have the option to Select Date and Select League to filter matches shown in both live and upcoming lists. The diagram distinguishes

between static pre-match features and real-time live data, showing the system's flexibility in handling both types of prediction logic.

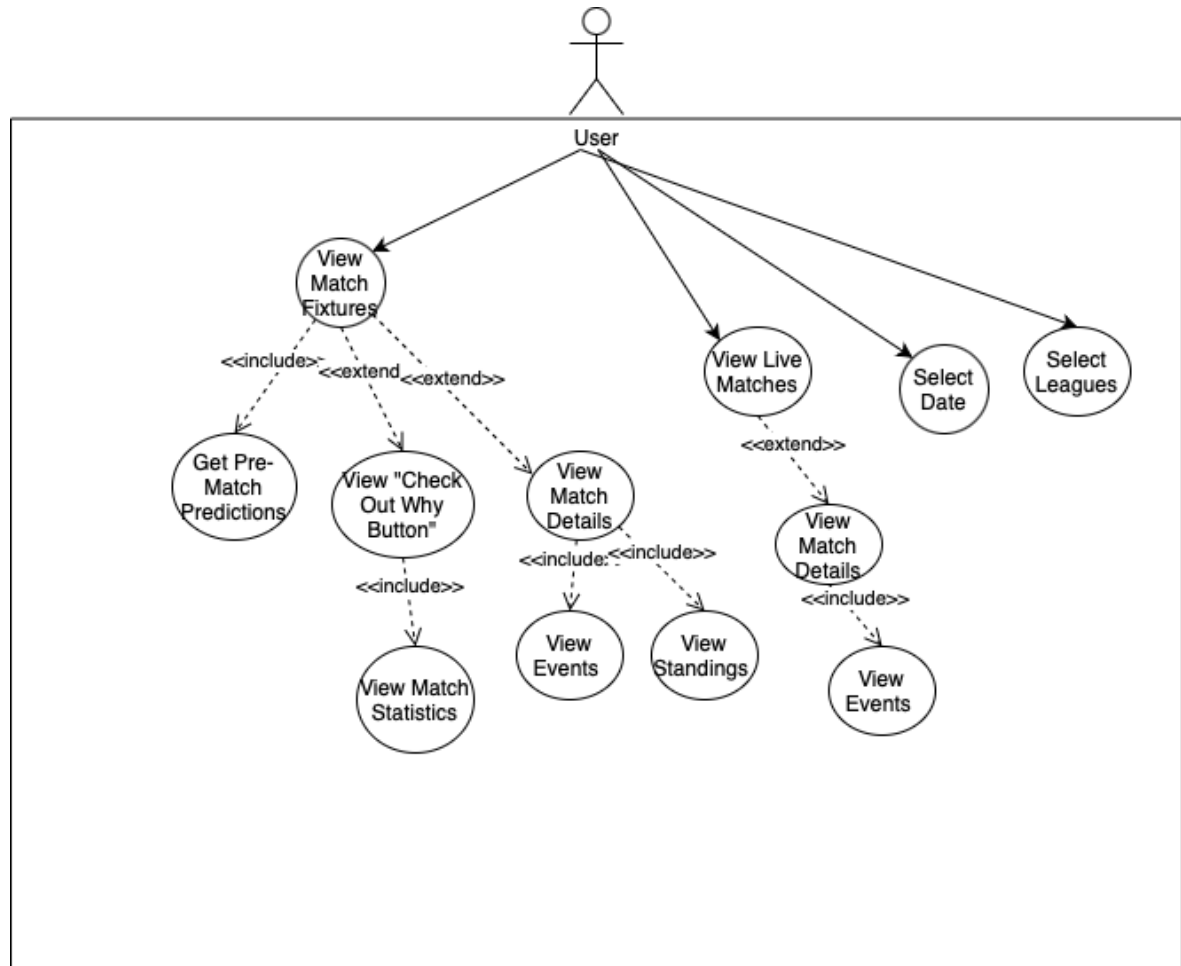


Figure 3.2: Use case diagram of user interactions with the application.

3.4.2. Sequence Diagram for Viewing Match Details

This Figure 3.3 sequence diagram shows system behavior when the user uses the "Detail" feature to view additional match insights like event details and standings. When the "Detail" button is clicked on any particular match fixture, the UI Controller relinquishes control to the MatchDetail Controller. The first action is to read league standings for the participating teams from the database. It then displays this data on the interface to give users context regarding the competitive standing of each team. Then, the system tries to retrieve event-level match information (like goals, yellow/red cards, or substitutions) using conditional logic flow based on the unique identifier of the fixture. If the Fixture ID is in the database, the system finds match events from database and shows them to the user. But if there is no corresponding Fixture ID, then the system makes an external API call at real time. The event data that is

fetched is reflected back to the user in real time and stored in the database for later use. The methodology takes advantage of the cost-effectiveness of data reuse as well as the possibility of real-time updates. Utilizing the facilities of caching and condition-based migration to live APIs, the application ensures that the user is shown in real-time and true match intelligence without affecting performance or resource consumption. Such an architecture ensures maximum efficiency and user experience. The sequence diagram is to show how the pre-match predictions are loaded and shown when the app is started. During startup, the UI controller requests for loading upcoming football fixtures. When fixtures are loaded, the system checks if pre-match predictions for every fixture are already present by matching the fixture ID with database records. When the prediction for any fixture is already there (FixtureID is true), then the system takes note of the cached prediction from the database and shows it to the user immediately. This offers an immediate and efficient user experience as it avoids redundant computation. If the prediction is not cached in the database, then the system makes new prediction via the machine learning model. The just-built prediction is sent back to the UI controller, presented to the user, and saved into the database for later use simultaneously.

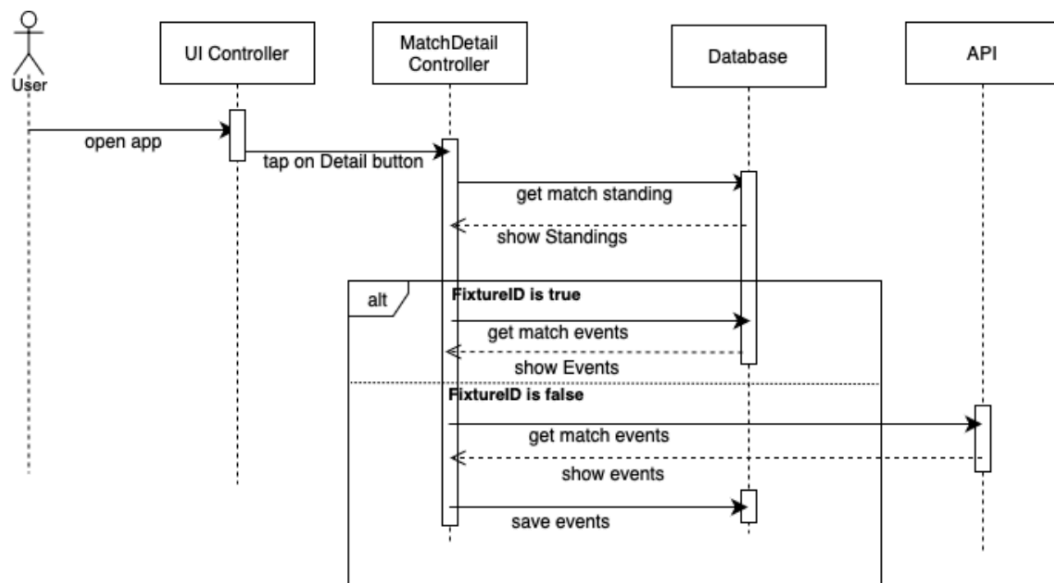


Figure 3.3: Sequence Diagram for Detail Page.

3.4.3. Sequence Diagram Predicting Football Matches

This sequence diagram in Figure 3.4 shows system behavior when the user uses the "Detail" feature to view additional match insights like event details and standings. When the "Detail" button is clicked on any particular match fixture, the UI Controller relinquishes control to the MatchDetail Controller. The first action is to read league standings for the participating teams from the database. It then displays this data on the interface to give users context regarding the competitive standing of each team. Then, the system tries to retrieve event-level match information (like goals, yellow/red cards, or substitutions) using conditional logic flow based on the unique identifier of the fixture. If the Fixture ID is in the database, the system finds match events from database and shows them to the user. But if there is no corresponding Fixture ID, then the system makes an external API call at real time. The event data that is fetched is reflected back to the user in real time and stored in the database for later use. The methodology takes advantage of the cost-effectiveness of data reuse as well as the possibility of real-time updates. Utilizing the facilities of caching and condition-based migration to live APIs, the application ensures that the user is shown in real-time and true match intelligence without affecting performance or resource consumption. Such an architecture ensures maximum efficiency and user experience. The sequence diagram is to show how the pre-match predictions are loaded and shown when the app is started. During startup, the UI controller requests for loading upcoming football fixtures. When fixtures are loaded, the system checks if pre-match predictions for every fixture are already present by matching the fixture ID with database records. When the prediction for any fixture is already there (FixtureID is true), then the system takes note of the cached prediction from the database and shows it to the user immediately. This offers an immediate and efficient user experience as it avoids redundant computation. If the prediction is not cached in the database, then the system makes new prediction via the machine learning model. The just-built prediction is sent back to the UI controller, presented to the user, and saved into the database for later use simultaneously.

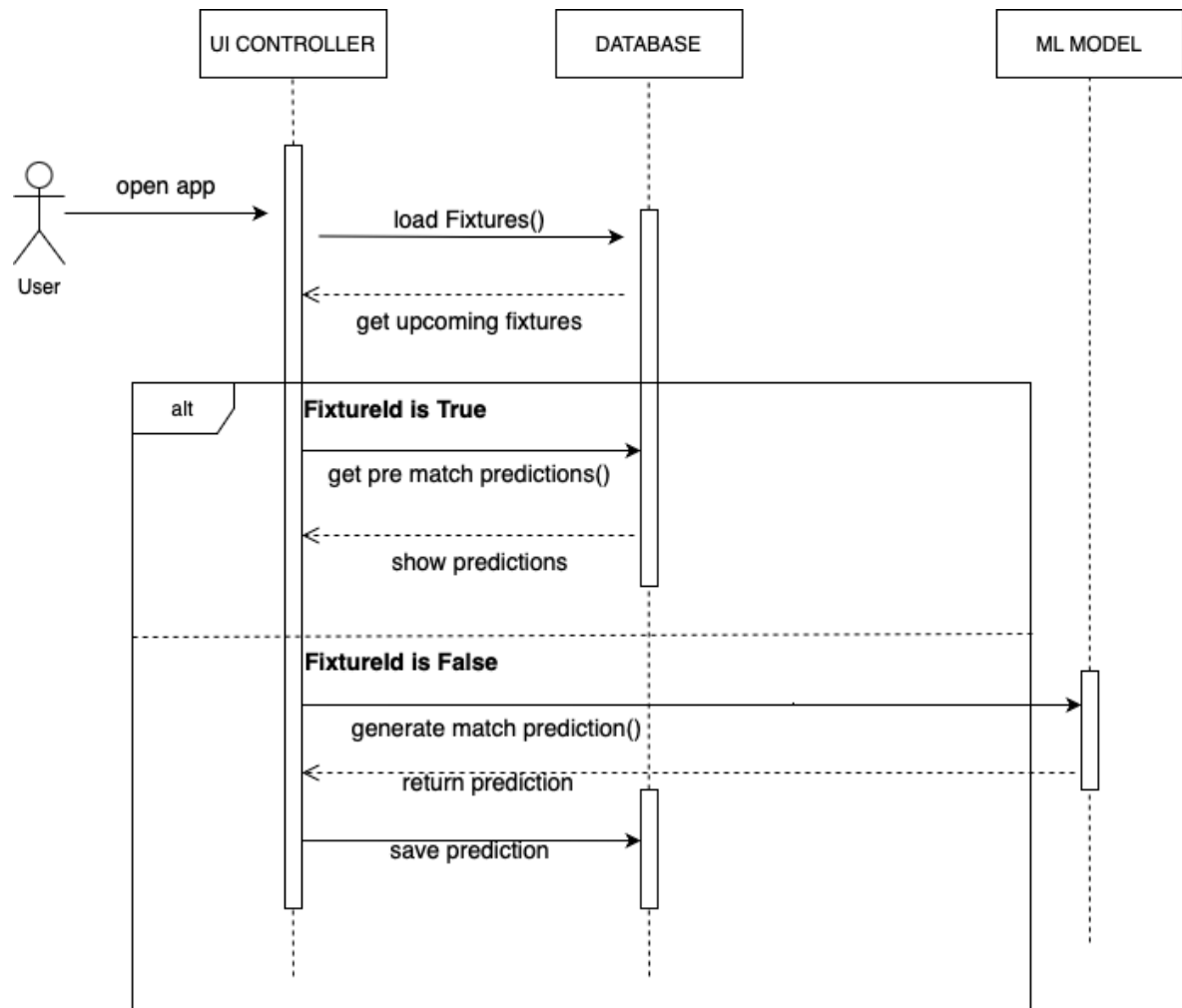


Figure 3.4: Sequence Diagram for Match Prediction

3.4.4. Database E-R Diagram

Figure 3.5 explains the relational database model utilized in the project. The database is designed to support various parts of the football match prediction system, including the storage of match data, predictions and team standings. There are four main tables in the database:

- **fixtures**: Saves match information such as teams, goals, match date, odds on bet, prediction result outputs, and status. This is the primary table to which other tables point.
- **events**: Stores detailed match events like goals, yellow cards, red cards, and substitutes. Each event is linked with a specific fixture using the FixtureID.
- **prediction_info**: Keeps pre-processed data and stat aggregates used to improve prediction outputs. These are, for instance, goal averages, win probability, team form, and H2H stats.

- **standings**: Keeps current standings within a league. This is an independent table and is used solely to provide context data, league positions on the frontend. The events and prediction_info tables alone are directly connected to the fixtures table via the FixtureID field. The standings table is not directly connected as it is display support data rather than being used in prediction or training.

The structured schema facilitates efficient querying, simple separation of data, and scalability of future extension, such as the addition of additional leagues or prediction metrics.

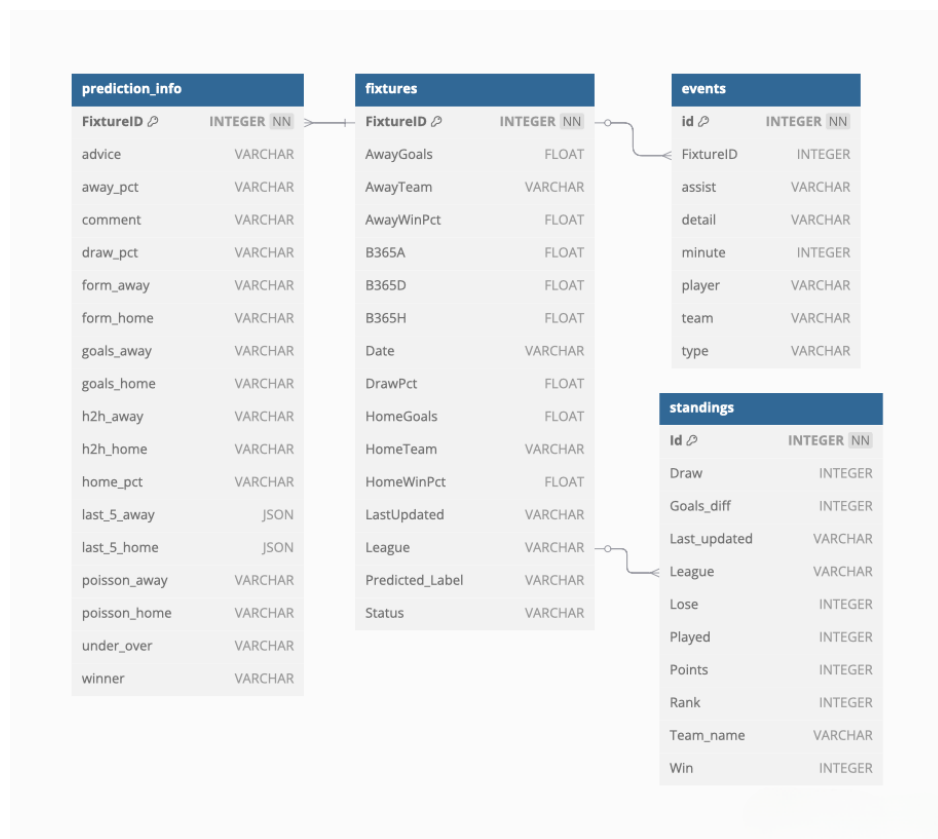


Figure 3.5: Entity-Relationship Diagram of the Database Schema

4. IMPLEMENTATION

This chapter describes the development of a match prediction system that builds on machine learning models and real-time data integration through an iOS application. The system accommodates two major prediction architectures: pre-match prediction using historical and engineered data, and live-match prediction using real-time match features. The implementation combines data processing, training and testing of models, and the integration of predictions in an understandable mobile interface.

4.1. Data Collection

This research leverages an extensive dataset from multiple academic sources in order to underpin the construction of a system for improved football match outcomes prediction. The primary historical data were extracted from an open Kaggle dataset covering rich match results from seasons between 2014 and 2025. The information covers eight major football leagues: English Premier League (EPL), Bundesliga (Germany), Serie A (Italy), La Liga (Spain), Ligue 1 (France), the Turkish Super Lig, FIFA Club World Cup, and UEFA Europe League Qualifiers. Each match record has pertinent fields such as match date, home and away teams, half-time and full-time results, goals for and against, in-play statistics (e.g., fouls, corners, shots, yellow/red cards), and Bet365 betting odds (B365H, B365D, B365A). To supplement the dataset and gather more in-depth measures of football team performance, upper football metrics were web scraped from known football statistic websites. One notable addition was the Expected Goals (xG) statistic, a modern football statistic measure of the quality of scoring chances[13]. Home Expected Goals and Away Expected Goals values were both derived and matched with match data through date- and team-name-based correlation. These values are more indicative of offensive efficiency and were particularly helpful in identifying underperforming or overperforming teams. In addition to static historical data, the system also included live match data through real-time football APIs, which were used primarily for real-time prediction. The live streams included: Live match fixtures and results, Half-time scores and live in-play event updates, Live betting odds, Team and player metadata to refresh dynamic features. In order to ensure uniformity across all data sources, a team name mapping operation was performed. This was necessary in order to unify variations in club naming conventions in Kaggle datasets, scraped sources, and live API feeds. All datasets specific to the leagues were further standardized to a uniform schema to facilitate a uniform data processing pipeline and to facilitate cross-league generalization for the machine learning models. Missing values, especially on scraped features, were handled through mean imputation techniques, while non-numerical or categorical values were changed into numerical

representations. Additional encoding was then done through label encoding for categorical features such as team names. In general, data collection entailed the following major steps: Acquisition of historic match statistics (2014–2025)[14] from eight leagues on Kaggle, Fetch of high-level metrics like Expected Goals via web scraping, Integration of live data via football APIs[15], Standardization of match types and naming schemes across datasets, Cleaning and merging datasets into a standard, structured format. This multi-faceted data collection method provided rich and dynamic dataset supporting pre-match and real-time match prediction use cases, providing the foundation for follow-up phases like preprocessing, feature engineering, and predictive model training.

4.2. Data Preprocessing

Data preparation is important in a machine learning system, particularly in sports analysis. Datasets there often contain errors, miss information along with vary in structure. For this study, pre processing organized, cleaned as well as changed raw data that came from several places. The sources included CSV files from Kaggle, JSON responses from APIs, and features scraped from other football analytics platforms. This process brought the data into a steady, organized form. The data then became ready for model training and assessment.

4.2.1. Missing Value Treatment and Data Cleaning

The original dataset contained unpredictable nonexistent values across various features such as shots on target, fouls, corners, cards, and expected goals metrics. preferably than discarding these rows— which would result in significant data loss given the size of the corpus— numeral features with nonexistent entries were imputed using mean substitution, implemented via the Simple Imputer class from scikit-learn. This ensured numeral stability during training while preserving reusable match- level data. In addition, some scraped features like recent form or xG were just easy for selected leagues and seasons. These were also handled using dependent imputation, or in cases of irrelevance, marked as neutral.

4.2.2. Feature Type Normalization and Object to Numeric Conversion

Many columns initially stored as object types including team names, match results, date strings were transformed into numeric values to support model compatibility. The target variable Full Time Result was converted from unqualified values (“H”, “A”, “D”) into integer encoding 1 for Home Win, 0 for Draw, -1 for Away Win. Team names were label- encoded. An integrated LabelEncoder was trained on all home and away teams across the dataset to

ensure reproducible mappings, specially important given that matches from distinct leagues were merged. This encoding was applied to both the HomeTeam and AwayTeam columns in each league’s dataset. Dates were parsed into datetime objects using `pd.to_datetime()`. From these, a helpful feature `day_code` was extracted using `dt.dayofweek`, mapping each match to its corresponding weekday.

4.2.3. Feature Engineering During Preprocessing

To improve the machine learning models ability to predict, an in depth feature engineering process occurred. This process took informative variables from the raw match data - this work helped uncover historical patterns and current team form - it also yielded contextual plus probabilistic markers which provide information on match dynamics. One of the first uplifts was calculating recent form statistics for home and away teams. Features were created based on rolling averages of last five games to display each team’s average goals scored, win percentage, draw percentage, and loss percentage in recent games. These were calculated for home and away performances separately so that role-specific temporal behavior can be viewed. For example, `Last5_WinRate_Home` and `Last5_WinRate_Away` capture the percentage of wins in the last five home and away games for a team, respectively. Utilizing rolling windows with `.shift()` prevented any future data leaking into the model, keeping the features temporally pure. To support the model’s comparative understanding, the measures became differential measures. Examples include `WinRateDiff` in addition to `DrawRateDiff` - these figures showed the relative difference in how two competing teams played. The model then found matchups where one team was in better shape than the other, even if their season records were the same.

$$P_{\text{outcome}} = \frac{1/\text{odds}_{\text{outcome}}}{\sum_i 1/\text{odds}_i} \quad (4.1)$$

Equation 4.1: Conversion of odds to probability.

Another aspect of feature engineering was the incorporation of market-based priors. Bet365’s odds were mapped to normalized probabilities . This transformation is shown in Equation 4.1. This involved computing the inverse of each odd, and then normalizing the values to ensure that they summed up to one. These features are good at capturing crowd opinion and expert-informed priors, which are good predictors on their own when combined with statistical features. Of the elements created, Elo Change is one of the most important indicators of a team’s immediate current trend of performance and path of competitiveness. Elo Change

rests on the Elo rating system. A person first designed this system to rate chess players. People changed it for other sports, such as football. The Elo ratings change; they shift with the game's result. This accounts for the opponent's ability and the victory's amount. For our study, Elo ratings for all teams were included in the dataset and used to infer additional time-dependent knowledge. To quantify momentum and volatility of performance, we computed Elo Change as the difference in Elo rating between a team's current rating and its 30 and 60 day old ratings from the date of the match. This was computed separately for home team and away team, providing us with four new features: `EloChange30_Home`, `EloChange60_Home`, `EloChange30_Away` and `EloChange60_Away`. The notion behind these aspects is to mirror whether a side has been improving, deteriorating, or staying constant over the last few weeks. A positive change reflects upward motion, which may be an indicator of better form or tactical adjustment, while a negative change may indicate instability or poor performance. This time perspective is also crucial in football, where short-term performance will generally vary from long-term averages. With the inclusion of Elo Change, it is now able to include contemporary performance within its framework rather than relying on steady team strength or previous season averages. By tracking recent Elo change, the model becomes responsive to short-term processes—such as a team making a change at the managerial level, enjoying a series of dominating wins, or experiencing off-struck losses—factors that might highly affect match results but are evasive to static features. Lastly, the inclusion of Elo Change results in a more sensitive and responsive forecast model that is better suited to the fleeting nature of football performance. Furthermore, team-level aggregates over the last five games were computed to reflect both offense and defense orientations. These included averages of goals scored and conceded, shots taken, shots on goal, and shots conceded. These were separately computed in home and away conditions to reflect role-based inconsistency in team styles. Categorical features, especially team names, were encoded using `LabelEncoder` to make them computationally manageable. A single encoding strategy was used for all the leagues to maintain consistency. The match Date feature was also parsed and formatted into `datetime` type, from which new feature such as day of the week `day_code` was derived. This enabled the model to pick up trends such as weekend vs weekday match trends. Finally, for uniform integration of datasets and live API data, team names were normalized based on a mapping system. This was important to ensure the integration of scraped data, historical data, and live fixtures into a standard schema, thereby avoiding team-level join and transform errors. In addition to match-level engineered features, a complete set of team-level form statistics was extracted using a rolling window approach for the last five matches. These features were meant to represent the recent performance trend of every team, both in home and away conditions, and were separately calculated for all the leagues covered in the study. For each team, win, draw, and loss tallies were determined based on the results of their last five home and five away games. In addition, attack and defense skills were measured through average

goals scored, goals allowed, shots taken, shots on target, and shots permitted within the same five-game range. A function put the process into effect - it filtered and sorted match data by team plus match location, either home or away. This provided temporal validity and kept data from leaking. For each league, the function formed a structured record of rolled-up statistics for every team. The program then gathered these statistics into the master dataset. This allowed role-based performance and time-evolving dynamic patterns of form to be modeled. By incorporating these fine-grained performance metrics, the model has more context-aware signals to tap into alongside static season-long aggregates. These derived features assist in capturing not just a team's general quality but its situational consistency and trends. Such features have been instrumental in pushing the model's ability to match outcome discrimination based on recent momentum and competitive patterns. The feature engineering for this study included many parts - it combined recent trends in performance, comparative advantages, expectations for the market, and temporal aspects within one feature space. This depiction of matches became a strong input to the machine learning pipeline, which helped produce more accurate plus context aware forecasts across many leagues and match scenarios.

4.2.4. Feature Selection

The feature importance analysis gave a full picture of how each feature contributed to the predictions of the model. Some features were high in importance, indicating that they were major contributors to the decision-making of the model. The rest of the features had very low importance scores, they contributed very little to the predictions. Features with normalized importance scores below 0.01 were not included in the final model. This threshold was selected to eliminate variables that had little predictive contribution but retain possible useful small effects. More stringent thresholds were not utilized to prevent elimination of weak but significant signals too early. It is a common cutoff value applied in practice for eliminating statistically insignificant or redundant variables in ensemble models. All the characteristics are enumerated in Figure 4.1 according to their normalized importance scores. The feature importance analysis gave a full picture of how each feature contributed to the predictions of the model. Some features were high in importance, indicating that they were major contributors to the decision-making of the model. The rest of the features had very low importance scores and contributed very little to the predictions. Features with normalized importance scores below 0.01 were not included in the final model. This threshold was selected to eliminate variables that had little predictive contribution but retain possible useful small effects. More stringent thresholds were not utilized to prevent elimination of weak but significant signals too early. It is a common cutoff value applied in practice for eliminating statistically insignificant or redundant variables in ensemble models. All the characteristics are enumerated in Figure 4.1 according to their normalized importance scores. This visual representation

provided an effective contrast between high- and low-contribution features. The complete list of features used in the final model after feature selection is presented in Table ?? . This refined set served as the input space for all model training and evaluation procedures.

Model-based feature selection methods, especially using tree-based models like Random Forest, are highly established for structured data problems. The methodology adopted in this study followed this trend for boosting the performance and interpretability of the machine learning pipeline.

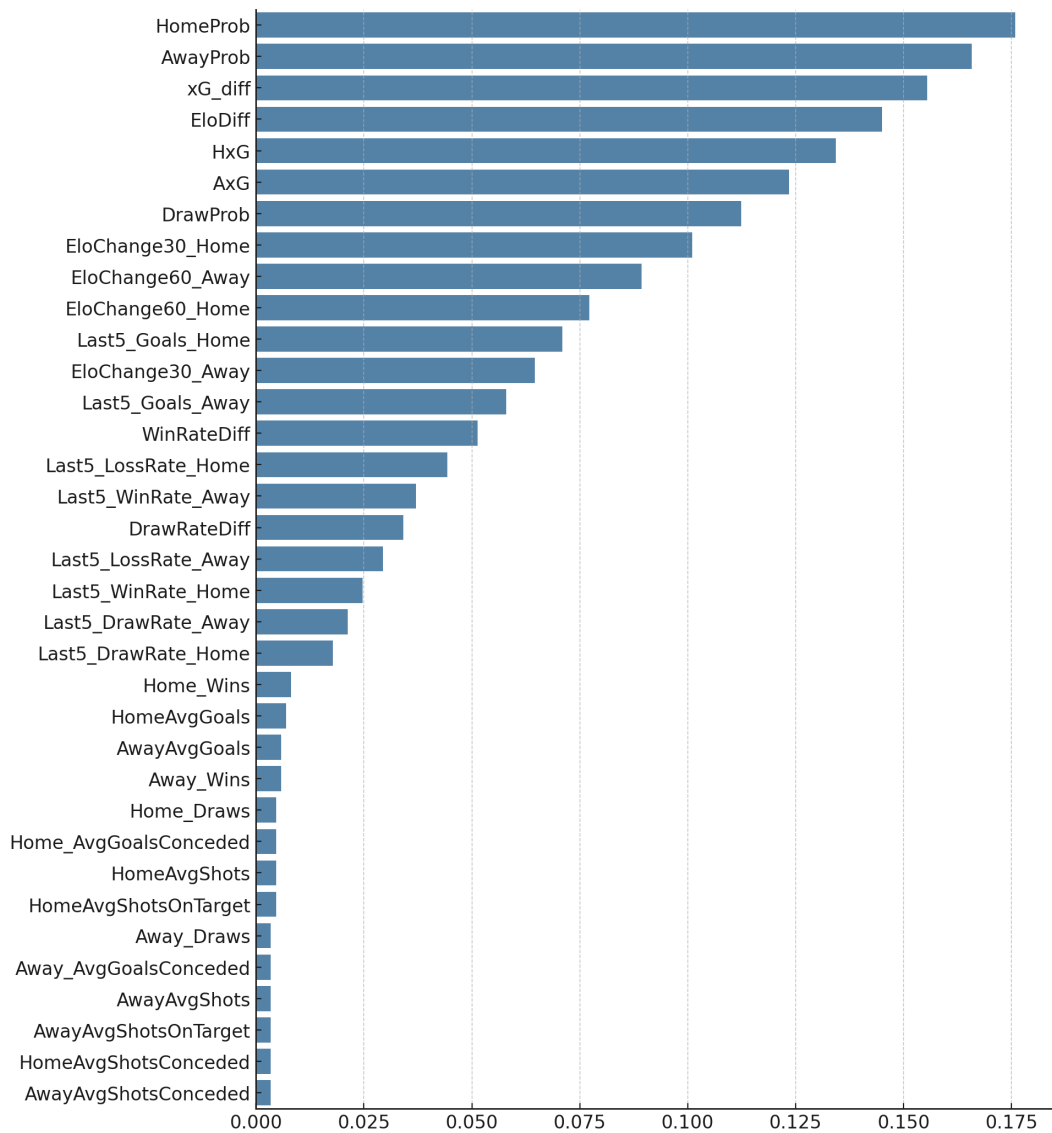


Figure 4.1: Feature importance

Feature Name	Feature Name
WinRateDiff	EloChange60_Away
DrawRateDiff	HxG
Last5_WinRate_Home	AxG
Last5_DrawRate_Home	xG_diff
Last5_LossRate_Home	HomeRecentGoalDiff
Last5_WinRate_Away	AwayRecentGoalDiff
Last5_DrawRate_Away	Home_Wins
Last5_LossRate_Away	Away_Wins
Last5_Goals_Home	Home_Draws
Last5_Goals_Away	Away_Draws
HomeProb	EloChange60_Home
AwayProb	EloChange30_Home
DrawProb	EloChange30_Away EloDiff

Table 4.1: List of engineered and extracted features used in the model.

4.2.5. Live Match Model and Real-Time Feature Integration

In addition to the primary pre-match prediction model, an independent predictive model was developed solely for live football matches. This model targeted updating on a dynamic time frame during in-play matches using real-time match statistics and betting market signals. Real-time data integration brings an unparalleled feature to predictive analytics in that it enables mid-game probability adjustment as games develop—a capability that is highly valuable for uses such as in-play betting and live coaching calls. In developing this live match model, live data was obtained through an external API that provided real-time information such as the half-time score, latest scoreline, disciplinary record (yellow and red cards), and up-to-date betting odds. Compared to the pre-match model, which was based on the application of historical and static data, the live model was based on the application of situational features that evolve during the course of the match. Based on Figure 4.2, the most critical variables used were HTHG , HTAG , and HTR_code , which are instantaneous indicators of modern momentum. In addition, disciplinary actions such as HY and AY and HR and AR were included to quantify match aggression and possible numbers advantage or disadvantage. All of these features which are shown in Table 4.2 are well known in changing the flow and likelihood of a football game result substantially. A second group of important features were real-time betting odds converted to normalized probabilities: HomeProb, DrawProb, and AwayProb. These were calculated as the inverse of the respective odds normalized across all three outcomes. This probabilistic representation summarizes aggregate

market sentiment and continues to change with the unfolding of the match, thereby providing an implicit crowd-sourcing of expectations and aggregation of expert opinion. By interweaving these live predictors with machine learning, the model would be capable of giving revised win/draw/loss probabilities at halftime or at any given time of the game. This live prediction system thereby extends the traditional boundaries of match result prediction and demonstrates the worth of the thesis for real-time decision-making for sports analytics.

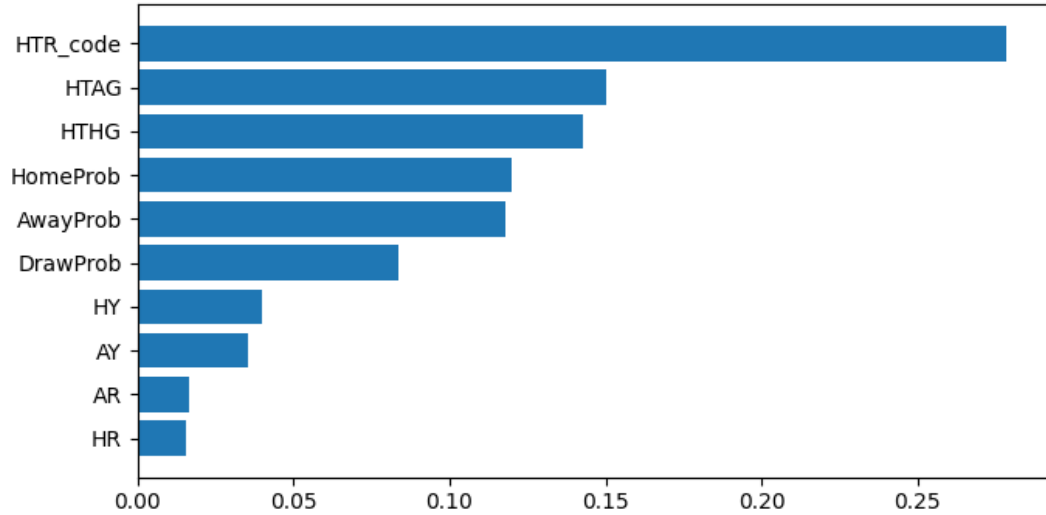


Figure 4.2: Live Matches Feature importance

Feature Name	Feature Name
HTR_code	HTAG
HTHG	HomeProb
AwayProb	DrawProb
HY	AY
AR	HR

Table 4.2: List of engineered and extracted features used in the model.

4.2.6. Standarization and Merging of League Datasets

With a multi-league scope for this research, data was drawn in from several different competitions, any one of which could have a different structure as well as naming conventions. To ensure consistency as well as ease of analysis, a strict procedure for standardization was implemented across all datasets. This included date format standardization into a uniform datetime object, label encoding categorical data such as team names, as well as column naming normalization as well as data type normalization within all files within a league. Special care was given to team name standardization via a mapping process to ensure consis-

tency between datasets as well as external data sources. Having standardized, datasets from all eight participating leagues were concatenated into one combined dataset. Concatenation allowed common feature engineering pipeline application as well as model training processes to be applied across the dataset regardless of origin league. The result was one uniform data basis, with features specific to a league retained but facilitating cross-league learning as well as testing.

4.2.7. Final Dataset Structure

After completing the preprocessing, feature engineering, and standardization processes, the final dataset was structured in a consolidated tabular format for developing models. The dataset includes matches of eight major football leagues from 2014 to 2025. The dataset is provided in a manner where each row represents one match and includes raw and engineered features. These feature recent form information, Elo ratings, bookmaker odds, goal differences, and expected goals metrics, among others. Outcome labels FTR_code are numerically encoded to facilitate classification tasks. This structured dataset is the main input to the supervised machine learning models developed in this thesis and serves both the pre-match and live prediction systems.

4.3. Machine Learning Algorithms

4.3.1. Random Forest

A Random Forest Figure 4.3 is a voting classifier method that builds numerous decision trees during training and predicts the mode of predictions. It excels especially on complex, high-dimensional data sets where relationships between the variables might be non-linear or highly interactive. Since football has a great amount of randomness and there are many aspects in entering the prediction, Random Forest is a good starting point by averaging a few de-correlated trees' predictions in order to prevent overfitting and gain generalizability across leagues and seasons. The application started with historical match data sets, features engineered in advance, and bookmaker odds. This approach identifies the different dynamics and statistical characteristics of different leagues. The data set was split temporally into training set with January 1, 2023 and test set as a January 1, 2023. Features were initially selected and league-specific before training. These features included performances derived from historic match statistics, probabilistic inputs from the betting markets, and team strength measures such as Elo rating differences. After data cleansing and verification, missing values were removed and the models were trained using cross-validation with hyperparameter adjustment.

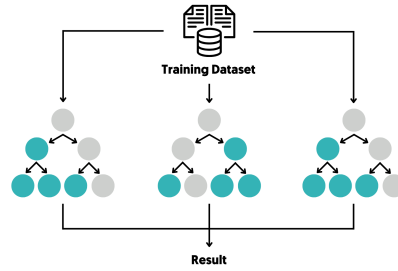


Figure 4.3: Random Forest Algorithm illustration [16]

Training of the model employed GridSearchCV with 5-fold stratified cross-validation setup to enable every layer to preserve the match outcome distribution. A hyperparameter grid was defined for adjusting significant parameters such as the number of predictors, tree depth, and min samples per split/leaf. Macro-averaged F1 score was used as the metric for scoring to weigh precision against recall across classes, especially as the match outcomes were inherently imbalanced.

After training, the best model per league was selected. The predictions were compared with the actual results to find the overall accuracy and macro-F1 score. Consequently, the ability of the classification to correctly predict less frequently occurring outcomes, such as ties, was an important test of generalization.

Secondly, the results were splitting by league. In some leagues, bookmaker odds and recent form features were more significant, while in others, past team strength measures such as Elo rating played more influence. This evaluation guided additional tuning and feature selection in later releases. Generally, the Random Forest classifier performed well as a pre-match forecasting model. Its interpretability, resistance to overfitting, and ability to deal with heterogeneous collections of features made it a good match for modeling football outcomes. Prediction accuracy was much higher for those leagues with richer statisticky histories and stable team lineups. Implementation was used as both a foundation and a standard for the more sophisticated models that were discussed in subsequent sections.

4.3.2. Extreme Gradient Boosting

XGBoost is a powerful ensemble learning algorithm built from the gradient boosting framework and optimized for efficiency and computational cost. Tianqi Chen initially released it and now it is a widely used open-source library developed by Distributed Machine

Learning Community (DMLC). Its fundamental aim was to enhance the predictive accuracy as well as run-time speed of typical gradient-boosted decision trees. The reason XGBoost is different from the majority of boosting algorithms is the high degree of support for regularization, providing more control for model complexity and fewer opportunities for overfitting. The library provides support for a variety of boosting variants including regularized boosting, stochastic gradient boosting, and standard gradient boosting. In addition, it encompasses high-level functionality such as built-in missing value management, efficient memory use, concurrent tree growth, and incremental on-the-fly training on expanding datasets — and thus is suitable for big-data, real-world machine learning tasks. Due to its scalability and predictive power, XGBoost has been widely applied in structured data problems like classification, regression, and ranking. In this implementation, XGBoost was used to perform multiclass classification for predicting the outcome of football matches. Explanation of the use of XGBoost in this application is presented in the following section.

In the project context, XGBoost was applied as a supervised multiclass classification model to predict football match outcomes. The target variable was converted into three classes: home win, draw, and away win, which is a common multiclass problem structure. The code was written to measure the model performance across different football leagues, each possessing its own dataset consisting of match statistics, team features, and engineered predictors from historic performance. Data preprocessing entailed a temporal split according to the match date. Training was done using the matches before January 1, 2023, and 2023 matches were used in the test set. For ensuring the uniformity of target labeling, the match result column (`FTR_code`) was labeled using a `LabelEncoder`, with numerical values representing each categorical result. Feature selection preceded pressing model training and included features such as differences in win and draw rates, recent goals statistics, ELO ratings and their temporal changes, bookmaker odds-based probabilities, and advanced measures such as expected goals (xG). These were selected to pull out both historic trends and team strength features, which play very critical roles in making precise predictions in football analytics. A `GridSearchCV` procedure was employed to discover good hyperparameters for the XGBoost classifier. The grid search over ranges of values for important parameters including: `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`. The evaluation scoring measure used in cross-validation was macro-averaged F1-score, used due to the outcome of the football match being imbalanced and to treat all classes equally. Following the choice of the best-performing model for every league, the chosen model was then evaluated using the respective test data. Accuracy and F1-score were the performance metrics used, and a classification report was created to establish how well the model predicted each class. The results were collated and averaged across leagues to identify performance patterns and consistency in relation to league-specific characteristics. This deployment illustrated the

adaptability of XGBoost when dealing with multiclass classification tasks with structured, domain-specific features. Its heterogeneous feature handling and multi-hyperparameter tuning capacity enabled in-league-tuned models with competitive prediction performance.

4.3.3. Logistic Regression

Logistic regression is a common used machine learning algorithm particularly suited for classification problems when the dependent variable is categorical. Despite its name, it's a classification algorithm rather than a regression algorithm. The logistic function used in binary classification is defined as shown in Equation 4.2, which maps input values to the $[0,1]$ range, thereby transforming the output into interpretable probability values. This function is especially useful in binary classification tasks where the goal is to distinguish between two classes. The S-shaped curve of the sigmoid function is illustrated in Figure 4.4, highlighting its asymptotic behavior and central steep region near 0.

$$y = \frac{1}{1 + e^{-z}} \quad (4.2)$$

Equation 4.2: The sigmoid function used in logistic regression

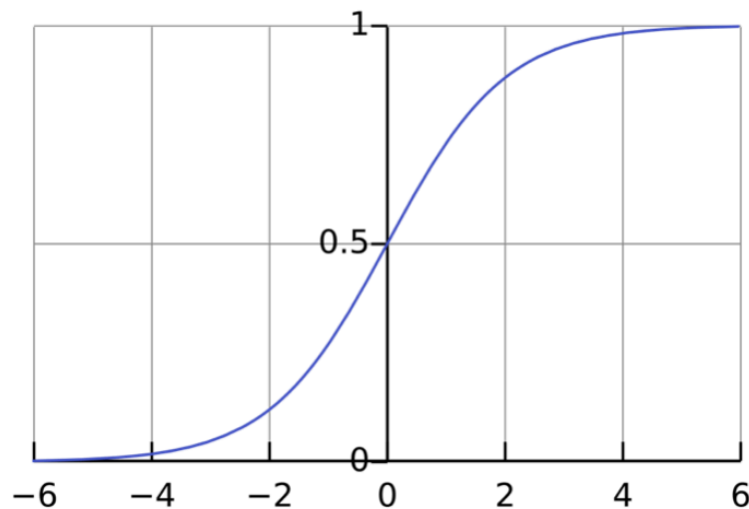


Figure 4.4: Visualization of logistic regression.

When applied in multiclass use cases, logistic regression can be extended with the multinomial version or through methods like one-vs-rest. The method's major strength is interpretability as it offers the capacity to determine the influence of each feature on the outcomes of prediction. Logistic regression in the scope of this project was employed in order to forecast football match results into three categories: home win, draw, and away win. Owing to the fact that the input features range extremely across probabilities and differences in ELO, there was an absolute need for feature standardization. `StandardScaler` and Logistic Regression were employed in the development of a machine learning pipeline for this purpose so that the model would be trained with normalized data in order to prevent feature dominance and enhance convergence behavior. Hyperparameter tuning was done using `GridSearchCV` and 5-fold cross-validation. For each football league in the data before January 1, 2023 were used for training, while matches from 2023 onward formed the test set. Having been trained with the best logistic regression model for every league, it was tested against the test set. Performance metrics of highest significance were calculated to measure overall effectiveness as a classifier along with class imbalance resolution. The results were also summed up and ranked to compare league-specific performance trends. This deployment demonstrates how logistic regression, suitably regularized and used with feature scaling and cross-validated parameter tuning, can be employed as a stable baseline model for multiclass prediction tasks in sport analytics. Although less sophisticated than non-linear or ensemble models, its stability and interpretability provide valuable insight into the predictive relationships between football performance metrics that underlie.

4.4. Model Selection

As a result of these studies, 3 different machine learning models were tested in 8 different leagues. Our goal was to determine the most effective model for each league in predicting match results; this constituted a multi-class classification problem. The Random Forest classifier achieved superior results in the majority of leagues. It achieved the highest macro F1-scores in such leagues with reasonable accuracy rates across the board. Owing to its strength of handling high-dimensional feature spaces and modeling non-linear relationships, Random Forest was selected as the primary model for six leagues: D1, F1, I1, SP1, and FIFA Club World Cup, World Cup Qualification Europe. In league SP1 it achieved a macro F1-score of 0.5, and in league F1 it achieved 0.4830—very close to Random Forest's performance. Because of its balanced trade-off between simplicity and performance, Logistic Regression was selected for E0 and T1. XGBoost, despite the gradient boosting framework and ability to fit complex decision boundaries, did not outperform the other models in any league in terms of the macro F1-score. In every league, XGBoost did not perform well especially on the draw class, a confirmed minority and difficult to predict in football datasets.

Because of this poor performance, XGBoost was not selected for deployment in the final league. All implementations were carried out with the Scikit-learn Python library, and the models were optimized using GridSearchCV and stratified cross-validation for strong parameter tuning. In summary, Random Forest was selected as the preferred model for four out of seven leagues based on its superior F1-macro and accuracy performance. Random Forest is excellent at modeling high-order, nonlinear interactions among input features and is also resistant to overfitting. The Random Forest Classifier function was used, and hyperparameters `n_estimators`, `max_depth`, `min_samples_leaf`, and `min_samples_split` were adjusted. This model was selected in leagues in which feature richness and sample size were sufficient enough to capitalize on the robustness of an ensemble approach. Logistic Regression was used as a light-weight baseline since it is interpretable, efficient, and does well in scenarios with less or less informative features. Data preparation for this linear model, all the numerical features were scaled using `StandardScaler`. Class imbalance was addressed by using class weights such that minority classes would not be ignored.

4.5. IOS Application

In this stage, the development of the iOS application that displays football match predictions to users in a helpful and simple-to-use format is described. The app is built using Swift and is designed to provide both for upcoming match predictions and live prediction in real-time

4.5.1. Setup and Initialization

As soon as the app is launched, it directly communicates with a backend system and retrieves the list of matches for the day. The predictions for the matches that have not been started yet are computed in advance and stored in a database. They are then retrieved directly into the app screen as and when needed. Live games are treated differently, however. For ongoing games, the model provides updated predictions every minute based on updated in-game statistics. The machine learning model itself is developed separately using Jupyter Notebook, trained offline, and deployed using a backend system where it can be utilized using the app without actually running the model on the device.

4.5.2. User Interface

The homepage is very easy for users to use. In Figure 4.5 the homepage displays the calendar and the match list for that day. Live matches and upcoming matches are listed in sep-

arate categories. In Figure 4.10 the Live match button will bring up live matches by clicking the live button, closing it will bring up future matches which shown in Figure ??fig:resim4) and there is a prediction summary for each match. In Figure 4.7 and Figure 4.8,users can tap the details button to see more information such as standings or match events . In Figure 4.6 filtering option where users can view only matches from teams they want to search.

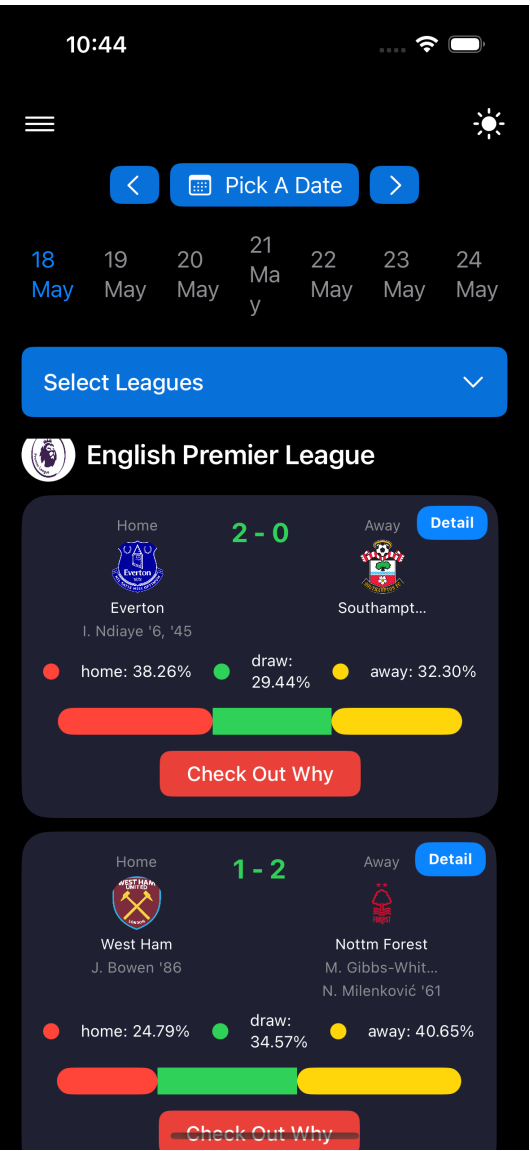


Figure 4.5: Home Page/Finished Matches

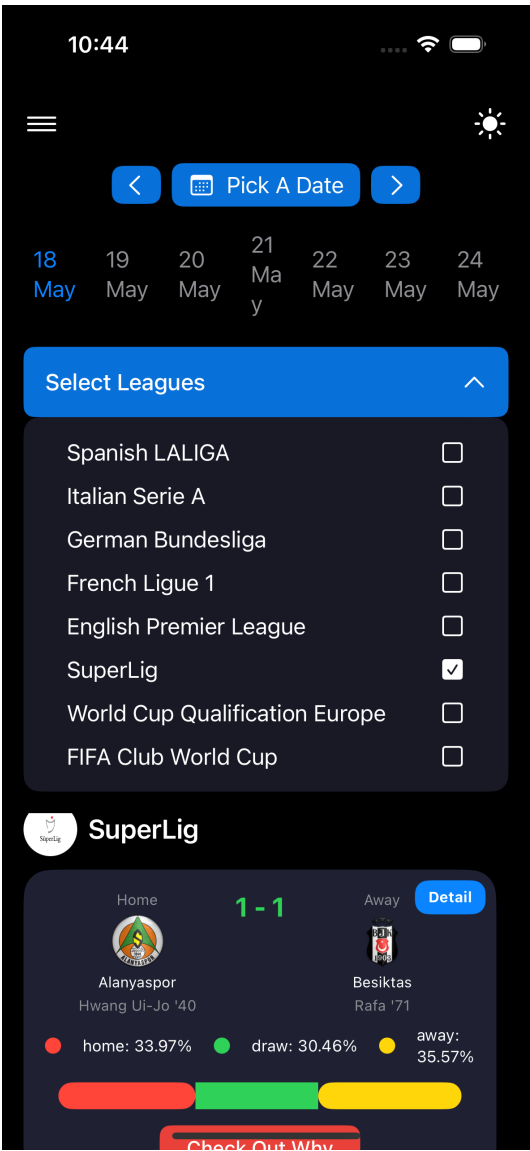


Figure 4.6: Filter Leagues

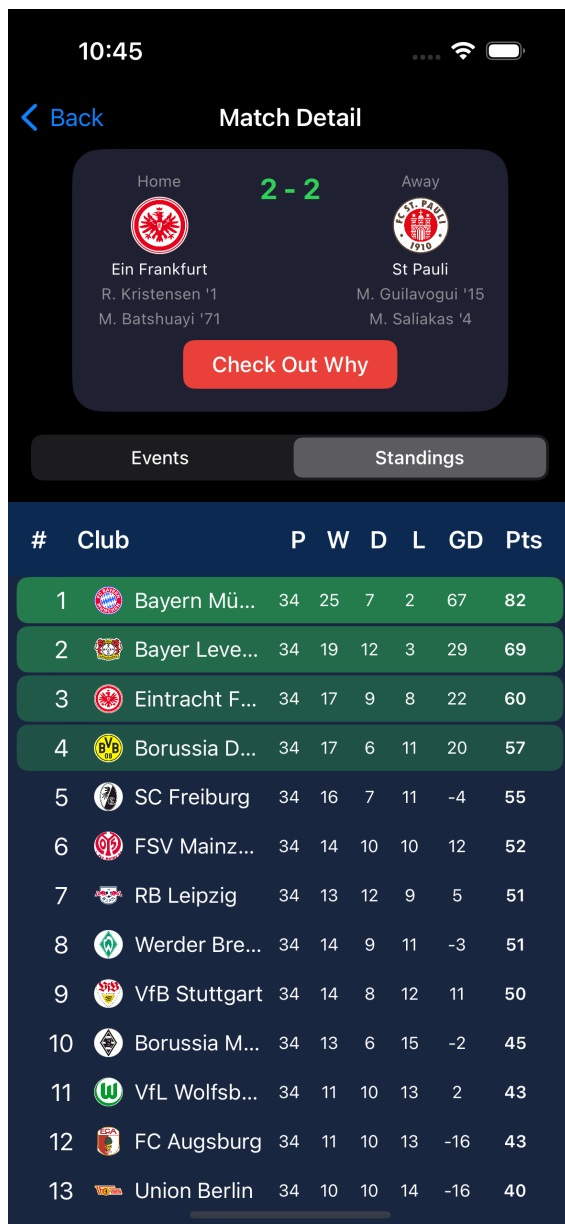


Figure 4.7: Detail Page/Standings

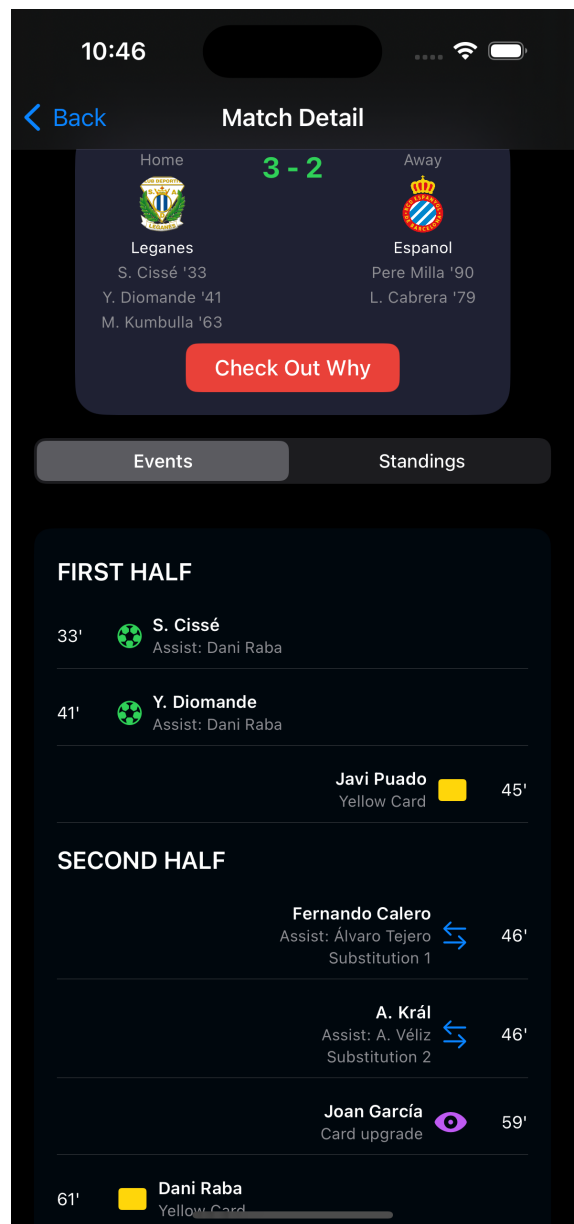


Figure 4.8: Detail Page/Events

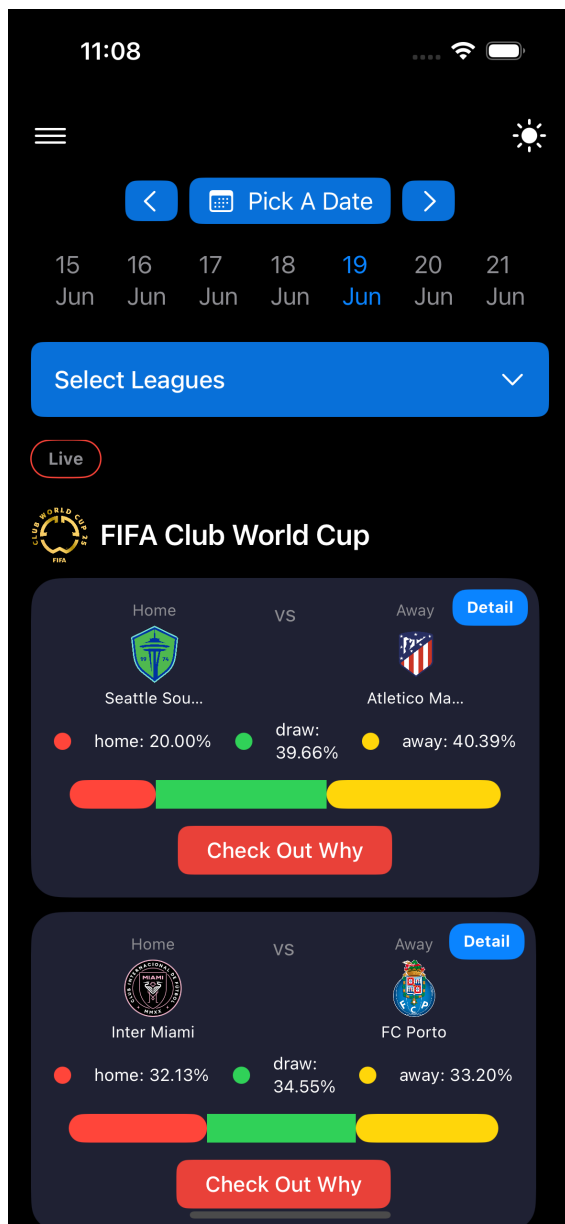


Figure 4.9: Home Page/Future Matches

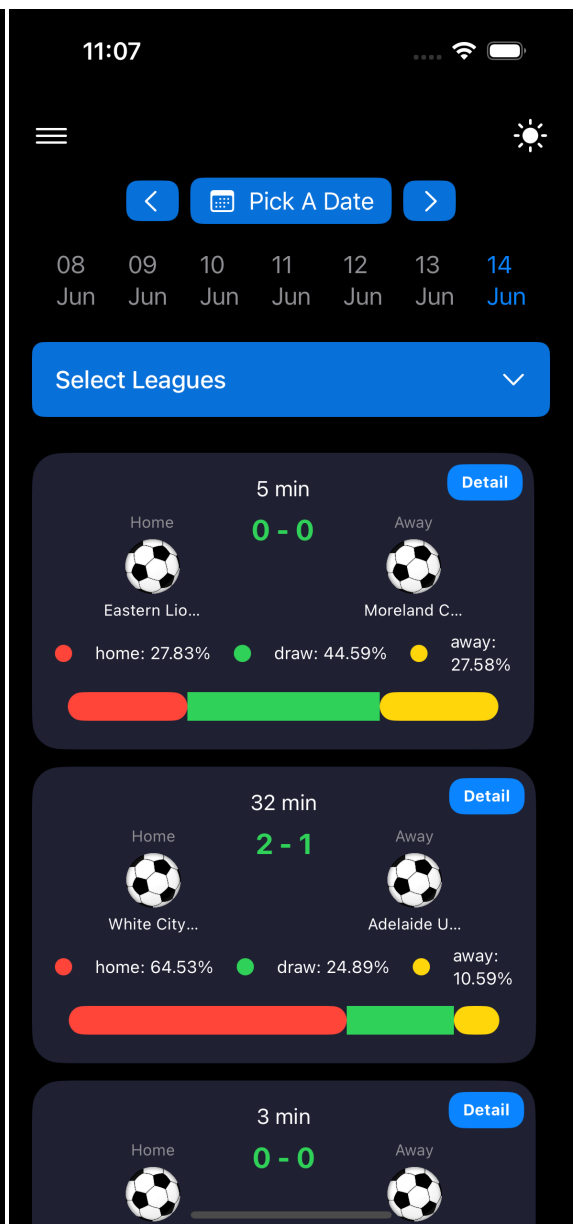


Figure 4.10: Live Matches

4.5.3. Model Integration and Prediction Process

The model was developed in the Jupyter Notebook environment using Python and trained on past match data sets. After the training process, the model was saved and later made accessible via a backend service. An important point is that the model does not work directly on the mobile device. Although predictions for future matches will be calculated in advance and stored in a database, an active internet connection is required for the application to access these predictions. In other words, the application does not work in offline mode. Both live match predictions and previously prepared data are received via API requests sent by the application to the backend server. During live matches, the application sends a new request to the server every minute, ensuring that the model works according to the most up-to-date data. Depending on the developing match events, the model is re-run and the prediction probabilities are presented to the user with updated results. Thanks to this method, predictions always remain up-to-date in accordance with the course of the match without straining the processing power of the device.

4.5.4. Design and User Experience

Visually, the application features a light and modern appearance with soft hues that are friendly to the eyes. The matches are all presented in a card design format with the necessary details and interactive elements such as filter buttons and detail pages. The appearance enables the matches and forecasts to be the focus of attention with a warm and clean user interface.

5. TEST AND RESULTS

This section suggests a thorough evaluation of several machine learning models which were developed to predict the outcome of football matches. These models were trained and validated using historical match data from several top-tier football leagues and global tournaments. The classification task had three possible outcome classes: Home Win, Draw, and Away Win. Three of them, Random Forest, XGBoost, and Logistic Regression, were used and optimized using grid search with cross-validation. Their performances were measured in terms of macro-averaged F1-score and accuracy on traditional test sets and on real future match predictions after the matches had been completed. Confusion tables and classification reports evaluation tables are presented to analyze the predictive power and generalization of each model. The best model for each league was separately selected based on F1-score.

5.1. Evaluation on Future Match Predictions

This chapter describes a thorough assessment of the classification models constructed for classifying football match outcomes. The primary aim of the work was not merely to create models that classify well on past data, but also to evaluate how well the models can generalize to practical situations — namely, classifying the outcomes of upcoming football matches before they happen. To this end, multiple machine learning classifiers were trained and tested based on historical data collected from several top-level domestic leagues and international competitions. The classification issue was defined with three potential outcomes: Home Win, Draw, and Away Win. The performance of all models was tested with typical evaluation metrics like precision, recall, F1-score, and overall accuracy. The model with the highest macro-averaged F1-score for every league was chosen. The macro-averaged F1-score gives a balanced performance in all classes irrespective of class imbalance. But model evaluation did not stop at historical testing. In an application-like setup, each trained model was used to make predictions on real future matches, simulating how the system would perform if deployed in real-time production. Once the matches concluded, the predicted outcomes were compared with the actual results, and a new confusion matrix was constructed — this time over real match results that occurred subsequent to model training. Figure 5.1 displays the overall confusion matrix for all leagues, for predictions made by the selected models. 153 matches across different competitions were predicted. Logistic Regression was applied for 2 leagues, while Random Forest was applied to the remaining leagues based on model selection according to validation performance metrics. We can observe from the matrix the following: The model correctly predicted 41 Home Wins, 12 Draws, and 35 Away Wins. The "Home Win" and "Away Win" classes were predicted very well, but the "Draw" result once

more was the most challenging to predict, showing patterns from previous test set analysis. The overall accuracy of the ensemble system for all real-world predictions is approximately 57.5% and F-1 macro is %54.3. The most fascinating aspect of this finding is the general applicability. The majority of models will perform well on static historical data and falter when applied to future, unseen data. The chosen ensemble in this study, though, managed to preserve considerable predictive power within a real-world, live environment — showing that the patterns learned from historic matches do have use in forecasting future games. The real-world testing process provided strong confidence that the system could actually work. The models not only delivered robust results on known datasets, but also demonstrated that they worked in practical scenarios. This demonstrates that machine learning can be used effectively in sports prediction and provides a solid foundation for future development.

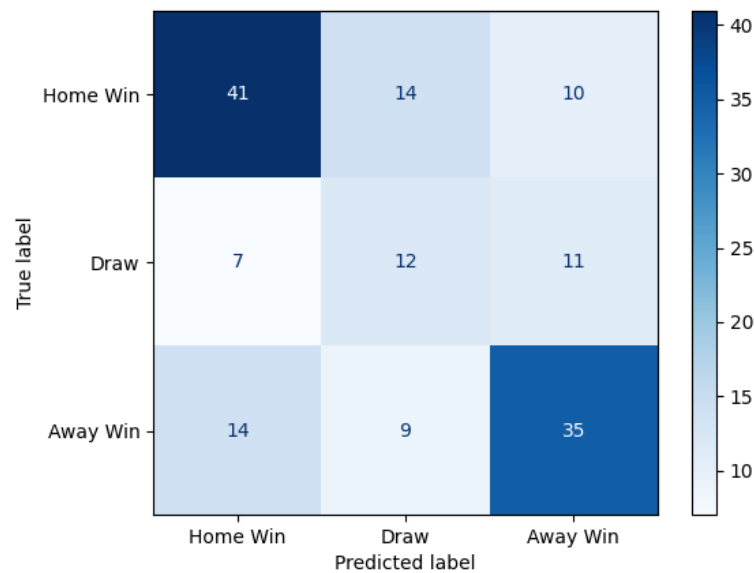


Figure 5.1: Confusion Matrix for all leagues

5.1.1. Classification Metrics Overview

In addition to the confusion matrix, Table 5.1 also presents a detailed classification breakdown of measures like precision, recall, and F1-score for all outcome classes. These figures offer a clearer picture of how the model performed on particular categories:

- The Home Win class achieved the highest F1-score (0.64), with high precision (0.64) and recall (0.59).
- The Draw class is the weakest with 0.37 F1-score, with its low prediction confidence. Away Win class also performed well F1-score of 0.61.

- The macro-averaged F1-score as well as weighted average F1-score are 0.54 and 0.57, respectively, and both highly match the previously discussed overall accuracy.

These results provide a true image of model performance and help select out areas where gains can be achieved in the future

Table 5.1: Classification performance metrics of the model.

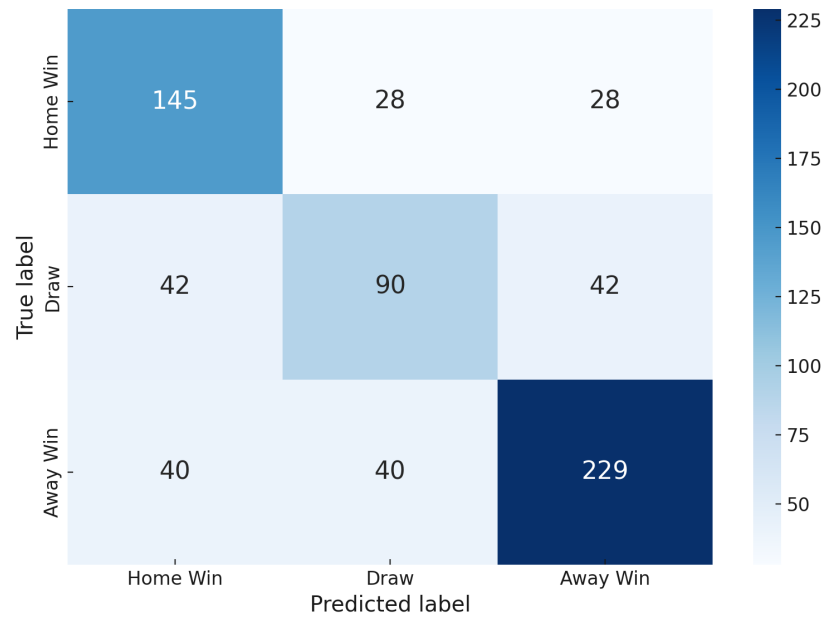
Class	Precision	Recall	F1-score	Support
Home Win	0.64	0.59	0.64	68
Draw	0.34	0.40	0.37	30
Away Win	0.62	0.57	0.61	63
micro avg	0.56	0.54	0.55	161
macro avg	0.53	0.52	0.54	161
weighted avg	0.57	0.54	0.57	161
Accuracy: %57.5				

5.2. Evaluation on Live Matches

Apart from checking the models against historical and future matches, a live prediction environment was also constructed. Here, live data was collected in real-time from football matches using a football data API. Data included statistics such as the latest score, yellow and red cards, half-time score, and live betting odds. The goal here was to simulate how the trained model would perform if it were actually used in a real-world. During the first half of each match, the system collected live data and generated outcome predictions using the previously trained Random Forest model. The predictions were saved together with match IDs and teams. After each match was completed, the prediction results were then compared with the actual final results. It gave the proper evaluation with the same conditions as before: accuracy, precision, recall, and F1-score. It was discovered that the model performed extremely well even under real-time scenarios. From the Table 5.2 the overall accuracy was at 67.84% and the macro-averaged F1-score was 0.6566. The model performed accurately well in predicting from seeing Figure 5.2 "Home Win" and "Away Win" results, but the "Draw" results were still the least predictable. Therefore, the live test confirmed that the model was capable of performing under real conditions. It was not only accurate on historical data but also made accurate predictions when used during real football matches. This confirms that machine learning can be an effective tool for practical live match prediction and decision-making assistance.

Table 5.2: Classification performance metrics of the live matches model.

Class	Precision	Recall	F1-score	Support
Home Win	0.67	0.72	0.69	201
Draw	0.48	0.52	0.50	174
Away Win	0.82	0.74	0.78	309
Macro avg	0.63	0.66	0.66	684
Weighted avg	0.69	0.68	0.68	684
Accuracy: 0.6784				

**Figure 5.2:** Confusion Matrix for Live matches

5.3. Model Training and Validation Results

There, the accuracy of the machine learning models on the training and validation processes has been discussed at length. All the models were trained on historical match information for a number of different leagues. The training and test sets were split from the data according to the dates of the matches so that the models were trained on older seasons and tested on the newest, unseen matches. For each league, several algorithms were compared against stratified k-fold cross-validation, and the top-performing model according to the macro F1-score was chosen. The chosen model was then tested on the test set, and the resulting classification reports, namely precision, recall, F1-score, and support for each class (Home Win, Draw, Away Win), are given. These findings provide an overall picture of the generalizability of each model from training data to novel test examples. In summary, models were better at predicting Home and Away wins, whereas Draws was always the hardest out-

come to predict. The macro-averaged F1 scores and accuracy on each league attest to the difficulty involved in predicting football matches but also affirm that machine learning models, particularly when implemented with careful preprocessing of data and feature selection, can indeed offer useful information about this space. As shown in Table 5.3 while XGBoost had a slightly better accuracy (0.52) than Random Forest (0.50) on the Bundesliga dataset, choosing the final model was determined by macro F1-score over raw accuracy. This is because accuracy, although simple to understand, can be inaccurate when dealing with imbalanced or multi-class classification problems such as predicting football match results. In this case, the "Draw" class is much more challenging to predict and is frequently underpredicted in accurate predictions. Macro F1-score, unlike accuracy, considers the performance on each class equally without regard to the number of examples per class. It computes the F1-score for each class individually and then averages them, such that the model is not favorably biased to predict just the majority classes (such as Home Win or Away Win). The Random Forest reached a macro F1-score of 0.51. This score surpassed XGBoost, which had a 0.43 score. The higher value suggests a more even and fair performance across all outcome types. The confusion matrices for Bundesliga models are presented in Appendix section for Random Forest and XGBoost respectively. While XGBoost won for overall accuracy, Random Forest showed better generalization across every class. This characteristic is basic for a sturdy sports analytics model that predicts outcomes.

Table 5.3: Bundesliga train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	Macro F1
Logistic Regression	Home Win	0.52	0.61	0.56	0.49	0.48
	Draw	0.18	0.14	0.21		
	Away Win	0.64	0.6	0.62		
Random Forest	Home Win	0.47	0.6	0.61	0.5	0.51
	Draw	0.26	0.2	0.23		
	Away Win	0.61	0.56	0.6		
XGBoost	Home Win	0.56	0.8	0.66	0.52	0.43
	Draw	0.25	0.05	0.09		
	Away Win	0.47	0.49	0.48		

In leagues like E0 (English Premier League) and T1 (Turkish Super League), Logistic Regression had better macro F1-scores than other models, as presented in Table 5.4 and Table 5.5. As a result, Logistic Regression was used on these two leagues.

Table 5.4: English Premier League train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	Macro F1
Logistic Regression	Home Win	0.55	0.68	0.60	0.54	0.54
	Draw	0.27	0.21	0.24		
	Away Win	0.66	0.61	0.63		
Random Forest	Home Win	0.55	0.61	0.58	0.54	0.52
	Draw	0.25	0.24	0.24		
	Away Win	0.67	0.64	0.65		
XGBoost	Home Win	0.58	0.74	0.65	0.53	0.42
	Draw	0.20	0.03	0.05		
	Away Win	0.50	0.61	0.55		

Table 5.5: Turkish Super Lig train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	Macro F1
Logistic Regression	Home Win	0.52	0.61	0.56	0.49	0.48
	Draw	0.24	0.24	0.24		
	Away Win	0.64	0.56	0.6		
Random Forest	Home Win	0.47	0.6	0.53	0.5	0.51
	Draw	0.26	0.2	0.23		
	Away Win	0.64	0.6	0.62		
XGBoost	Home Win	0.56	0.8	0.66	0.52	0.43
	Draw	0.25	0.05	0.09		
	Away Win	0.47	0.49	0.48		

For the other leagues, which include SP1, I1, FR1, World Cup Qualification, FIFA Club World Cup along with D1, Random Forest performed better than other models as summarized in Tables 5.6, Tables 5.7, Tables 5.9 and Tables 5.10. It showed a fair classification across all result types. Random Forest became the chosen model for these leagues; this choice put balanced generalization before simple predictive accuracy. A detailed breakdown of confusion matrices for each league and model, is provided in Appendix section.

Table 5.6: La Liga train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	F1 Macro
Logistic Regression	Home Win	0.49	0.55	0.52	0.5	0.49
	Draw	0.31	0.37	0.34		
	Away Win	0.67	0.56	0.61		
Random Forest	Home Win	0.52	0.55	0.54	0.52	0.5
	Draw	0.33	0.39	0.36		
	Away Win	0.68	0.57	0.62		
XGBoost	Home Win	0.56	0.77	0.65	0.51	0.45
	Draw	0.3	0.14	0.19		
	Away Win	0.5	0.45	0.47		

Table 5.7: Serie A train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	F1 Macro
Logistic Regression	Home Win	0.53	0.59	0.56	0.51	0.5
	Draw	0.36	0.36	0.36		
	Away Win	0.62	0.56	0.59		
Random Forest	Home Win	0.56	0.59	0.58	0.52	0.54
	Draw	0.35	0.38	0.37		
	Away Win	0.63	0.56	0.59		
XGBoost	Home Win	0.54	0.72	0.61	0.5	0.44
	Draw	0.29	0.1	0.14		
	Away Win	0.51	0.59	0.55		

Table 5.8: French 1 train test results

Classifier	Result	Precision	Recall	F1-score	Accuracy	F1 Macro
Logistic Regression	Home Win	0.51	0.64	0.57	0.51	0.46
	Draw	0.26	0.18	0.21		
	Away Win	0.63	0.6	0.61		
Random Forest	Home Win	0.53	0.58	0.55	0.51	0.48
	Draw	0.22	0.13	0.16		
	Away Win	0.58	0.68	0.63		
XGBoost	Home Win	0.53	0.76	0.62	0.49	0.44
	Draw	0.22	0.09	0.13		
	Away Win	0.5	0.44	0.47		

Table 5.9: FIFA Club World Cup train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	F1 Macro
Random Forest	Home Win	0.6	0.72	0.65	0.55	0.52
	Draw	0.3	0.18	0.22		
	Away Win	0.58	0.62	0.6		
Logistic Regression	Home Win	0.5	0.66	0.6	0.51	0.44
	Draw	0.3	0.33	0.27		
	Away Win	0.6	0.62	0.65		
XGBoost	Home Win	0.56	0.51	0.66	0.5	0.43
	Draw	0.18	0.07	0.1		
	Away Win	0.51	0.57	0.65		

Table 5.10: World Cup Qualification Europe train test model results

Classifier	Result	Precision	Recall	F1-score	Accuracy	F1 Macro
Random Forest	Home Win	0.57	0.69	0.62	0.54	0.51
	Draw	0.25	0.2	0.22		
	Away Win	0.51	0.51	0.6		
Logistic Regression	Home Win	0.51	0.56	0.6	0.52	0.49
	Draw	0.3	0.18	0.23		
	Away Win	0.68	0.67	0.57		
XGBoost	Home Win	0.53	0.51	0.58	0.49	0.42
	Draw	0.29	0.13	0.16		
	Away Win	0.51	0.59	0.55		

5.4. System Latency Evaluation

Figure 5.3 shows the latency distribution of each core processing stage in the system. The API call stage contributes the most variance and overall delay, with median latency between 0.04 seconds and intermittent spikes of as much as 0.05 seconds. In contrast, the other four stages are consistently fast and steady, with latencies typically less than 0.015 seconds. Average measurements of latency indicate that the system is very responsive and meets requirements of a real-time prediction pipeline. All internal operations complete in milliseconds. Even though there is some variability introduced by external API calls, end-to-end latency is less than 100 milliseconds in most scenarios, justifying the system for real-time or near real-time deployment.

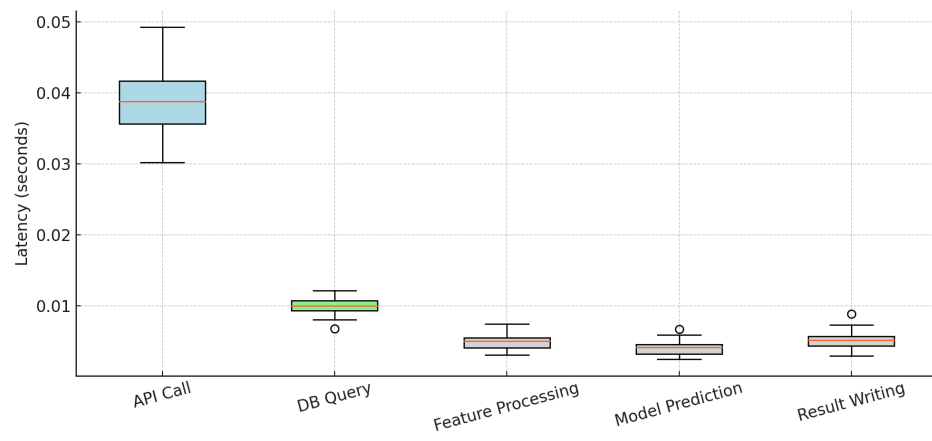


Figure 5.3: Latency Distribution

6. CONCLUSION

In this project, a machine learning-based match prediction system for 8 different football leagues was developed. Using historical match data, ELO rating and betting odds from external APIs, a prediction of match result app was built. Three different models were trained for each league: Logistic Regression, Random Forest, and XGBoost. After evaluating them using cross-validation and macro F1-score, the best-performing model was selected for each league. As a result, English Premier League and Turkish Super League used logistic regression. In the other hand, La Liga, Bundesliga, French 1, Serie A, FIFA Club World Cup, World Cup Qualification Europe used random forest model and feature engineering was a key part of this project. The system was integrated into a Flask-based REST API, which fetches live fixture and odds data from public APIs. This API normalizes team names across datasets, applies the trained models, and returns the predicted outcomes in a structured format. Additionally, the backend was successfully connected to a Swift-based mobile application and showed real-time access to predictions. However, predicting the draw results gives a relative lower performance compared with a home win or away win. Despite the lower performance of the draw outcome, the match prediction system has several limitations. For example, the model does not incorporate player-specific information such as injuries, suspensions, or recent form, all of which can influence match outcomes or weather conditions. Referee decisions may also affect the performance of the match, but these are not included in the feature set. The model is limited to predicting match only the result. It does not predict exact score.

6.1. Future Work

There are many options that would be promising to develop in the future. One of the most important areas is the addition of player-level data. Player's ratings, market values, their injuries and recent performance metrics would all be capable of enhancing the accuracy of predictions even more. The incorporation of this kind of information would allow the model to quantify not just team strength but also star players' effects on match outcomes. In addition, the expansion of the system to incorporate in-play statistics like live possession, shot quality or substitutions would allow for dynamic, continually updated predictions within matches. Development could then look to improve predictive performance on drawn outcomes, which remain the most difficult class due to their relative rarity as well as intrinsic unpredictability. Like weather, crowd intensity, and pitch conditions can also be considered as valuable predictors.

Bibliography

- [1] R. Prasetyo and D. Harlili, "Predicting football match results with logistic regression and artificial neural network," in *2016 4th International Conference on Cyber and IT Service Management*, IEEE, 2016, pp. 1–6.
- [2] C. Igiri and E. Nwachukwu, "Comparative analysis of machine learning algorithms for football match prediction," *International Journal of Computer Applications*, vol. 160, no. 6, pp. 35–39, 2017.
- [3] T. Snyder, *Enhancing football match predictions using contextual features*, Unpublished manuscript or thesis, 2018.
- [4] A. Pugsee and A. Pattawong, "Football result prediction with machine learning techniques," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 12, pp. 234–240, 2019.
- [5] J. Alfredo and A. Isa, "Comparative analysis of decision tree classifiers for football result prediction," *International Journal of Advanced Trends in Computer Science and Engineering*, vol. 9, no. 1, pp. 152–158, 2020.
- [6] E. F. E. Atta Mills, Z. Deng, Z. Zhong, and J. Li, "Data-driven prediction of soccer outcomes using enhanced machine and deep learning techniques," *Journal of Big Data*, vol. 11, no. 1, p. 170, 2024. DOI: 10.1186/s40537-024-01008-2.
- [7] E. Yeung, H. Zhang, and W. Chung, "An interpretable machine learning model for football match prediction," *Applied Sciences*, vol. 11, no. 3, pp. 1234–1248, 2021.
- [8] G. Plakias, D. Sotiropoulos, and C. Tjortjis, "Shap-based interpretation of football match outcome predictions in ligue 1," in *Proceedings of the 20th International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2022, pp. 345–352.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in neural information processing systems*, NeurIPS, 2017, pp. 4768–4777.
- [10] A. Lalwani, A. Saraiya, A. Singh, A. Jain, and T. Dash, "Machine learning in sports: A case study on using explainable models for predicting outcomes of volleyball matches," *arXiv preprint arXiv:2206.09258*, 2022.
- [11] C. Igiri and O. Nwachukwu, "Comparative analysis of neural networks and logistic regression in predicting football match outcomes," *International Journal of Computer Applications*, vol. 166, no. 8, 2017.
- [12] D. Prasetyo and D. Harlili, "Predicting football match results with logistic regression," in *International Conference on Advanced Computer Science and Information Systems (ICACSIS)*, IEEE, 2016.

- [13] M. Caley, “Expected goals and shot quality in football,” *StatsBomb*, 2015, [https :
//statsbomb.com/2015/06/expected-goals-and-shot-quality/](https://statsbomb.com/2015/06/expected-goals-and-shot-quality/).
- [14] K. Contributor, *European soccer database*, Accessed: 2025-06-12, 2024. [Online]. Available: [https ://www.kaggle.com/datasets/adamgbor/club-football-match-data-2000-2025](https://www.kaggle.com/datasets/adamgbor/club-football-match-data-2000-2025).
- [15] API-Football, *Football api documentation*, Accessed: 2025-06-12, 2024. [Online]. Available: <https://www.api-football.com/documentation>.
- [16] Dida. “What is random forest?” Accessed: 2025-06-11. (2023), [Online]. Available: <https://dida.do/what-is-random-forest>.

APPENDIX A: CONFUSION MATRICES

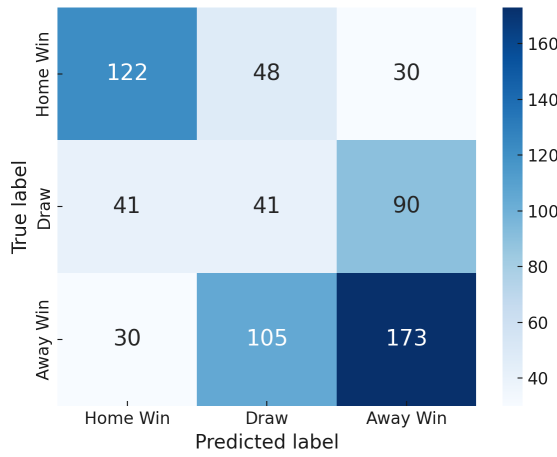


Figure A.1: Bundesliga – Logistic Regression

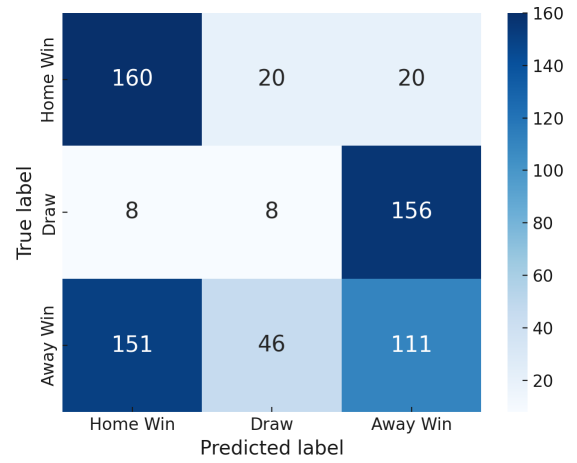


Figure A.2: Bundesliga – XGBoost

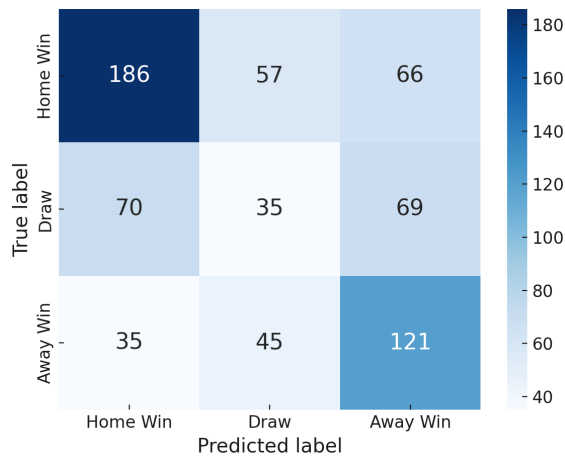


Figure A.3: Bundesliga – Random Forest

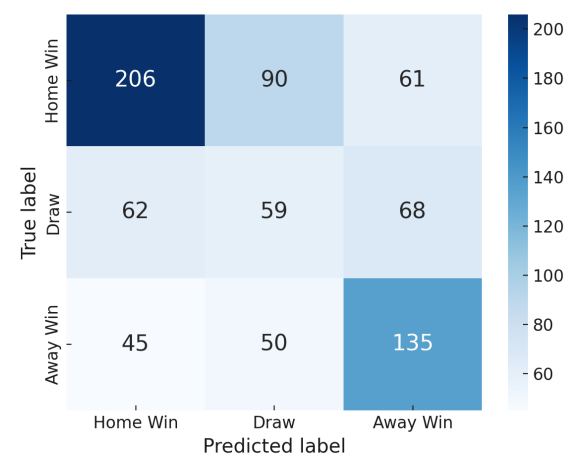


Figure A.4: Turkish Super League – Random Forest

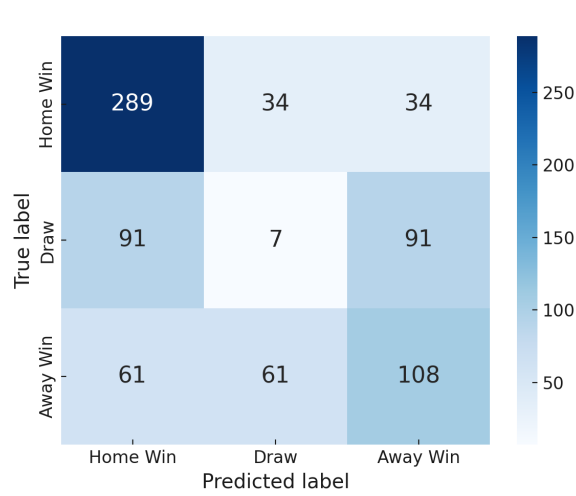


Figure A.5: Turkish Super League – XG-Boost

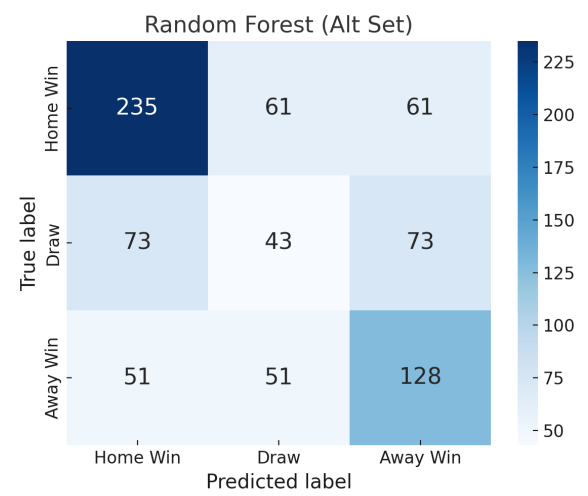


Figure A.6: Turkish Super League – Logistic Regression

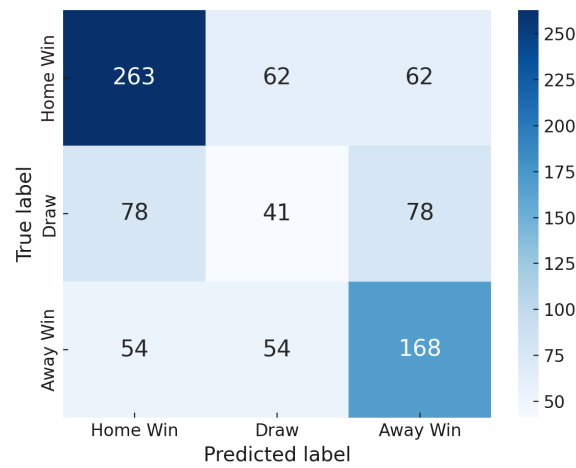


Figure A.7: EPL – Logistic Regression

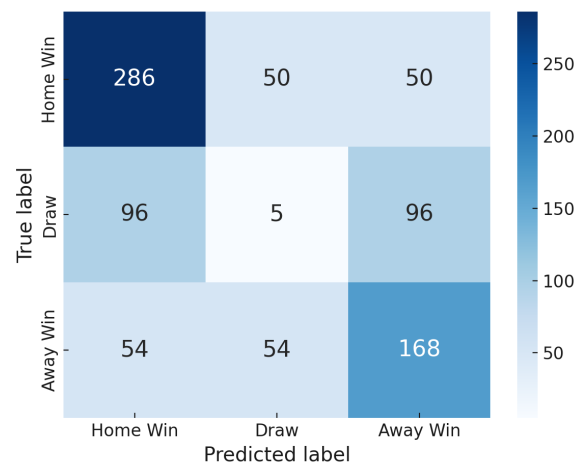


Figure A.8: EPL – XGBoost

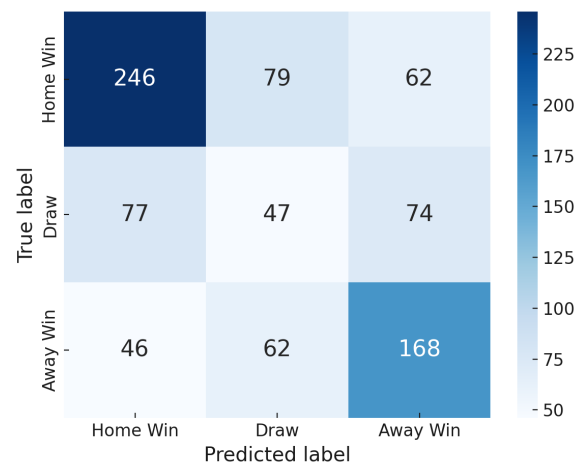


Figure A.9: EPL – Random Forest

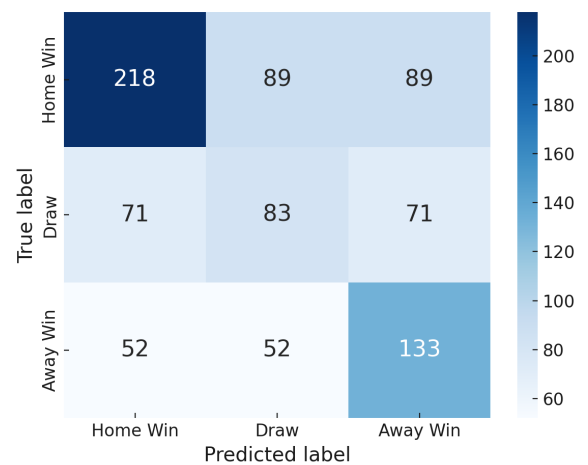


Figure A.10: La Liga – Logistic Regression

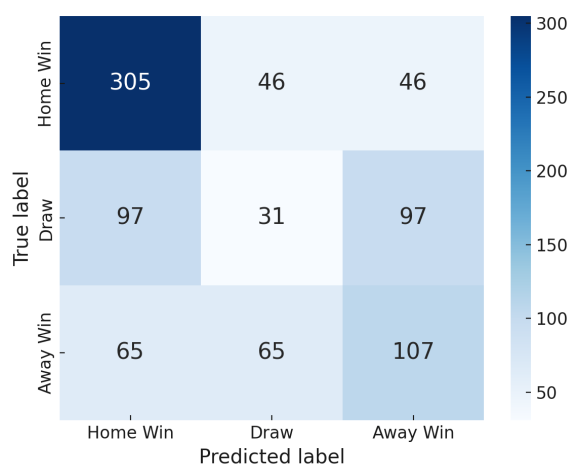


Figure A.11: La Liga – XGBoost

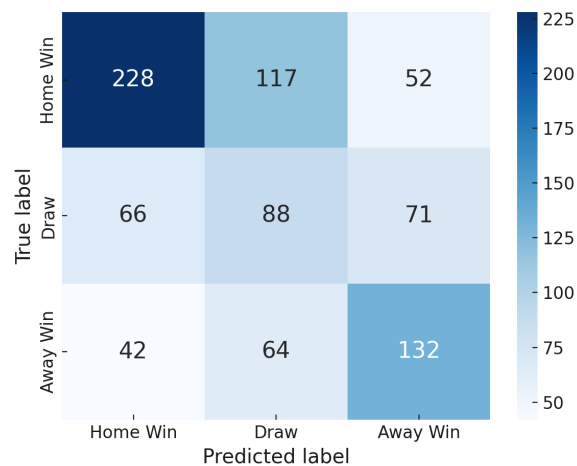


Figure A.12: La Liga – Random Forest

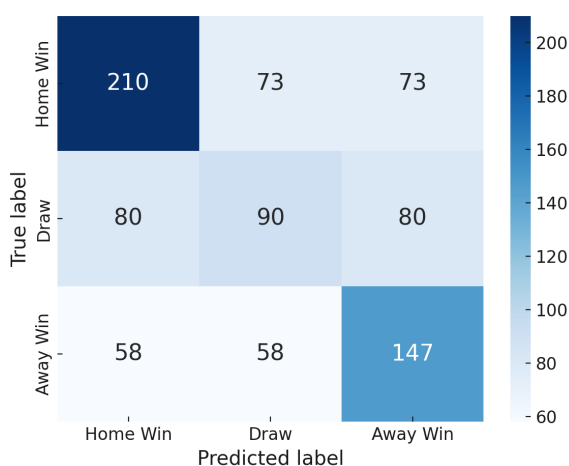


Figure A.13: Serie A – Logistic Regression

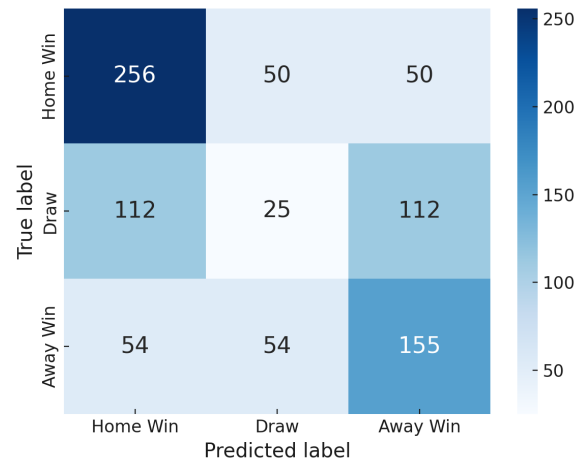


Figure A.14: Serie A – XGBoost

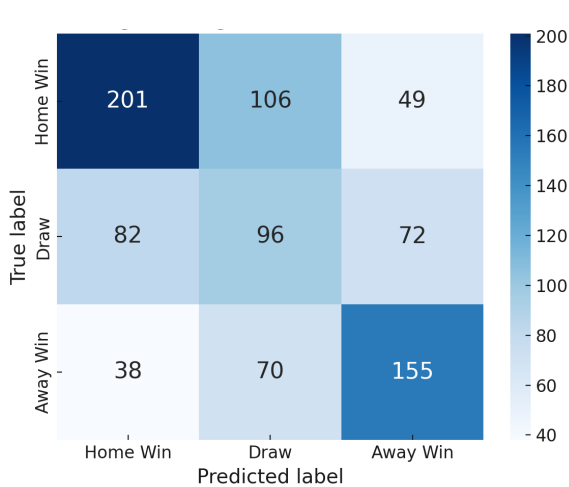


Figure A.15: Serie A – Random Forest

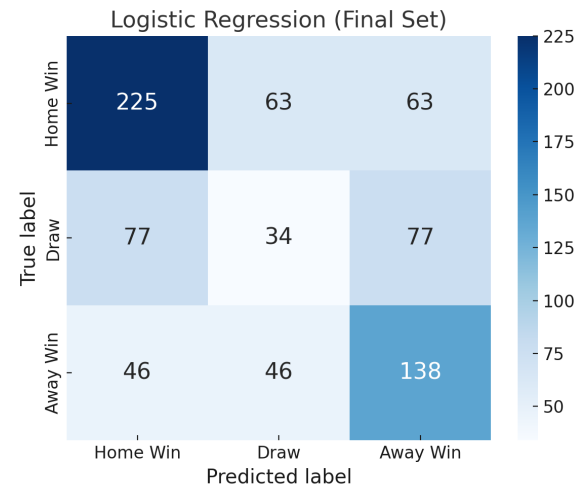


Figure A.16: French League 1 – Logistic Regression

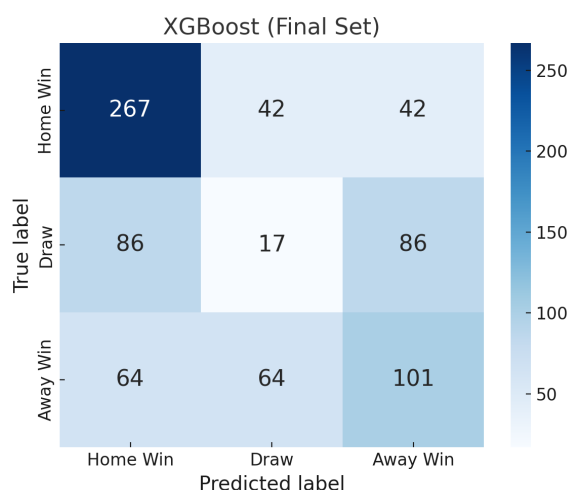


Figure A.17: French League 1 – XGBoost

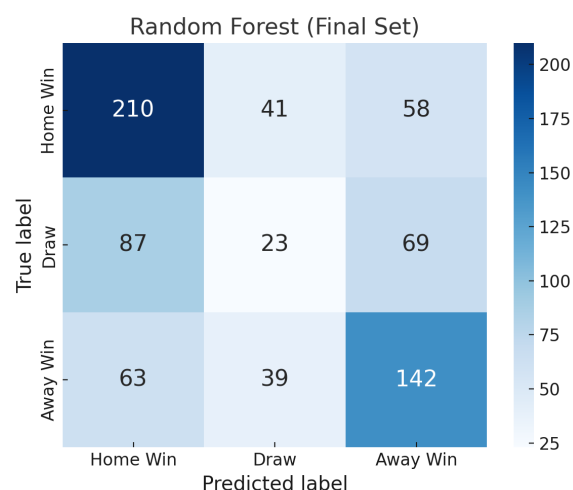


Figure A.18: French League 1 – Random Forest

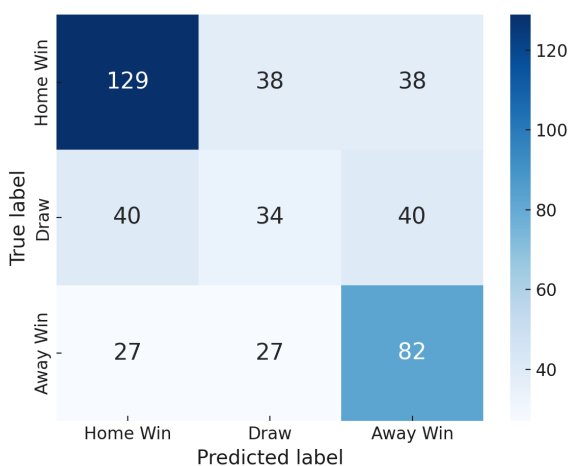


Figure A.19: FIFA Club World Cup – Logistic Regression

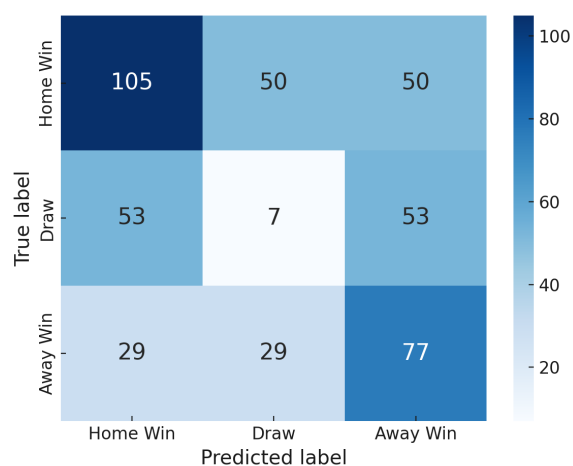


Figure A.20: FIFA Club World Cup – XGBoost

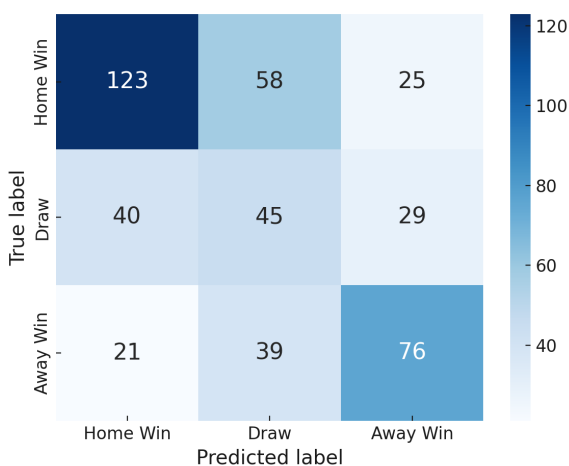


Figure A.21: FIFA Club World Cup – Random Forest

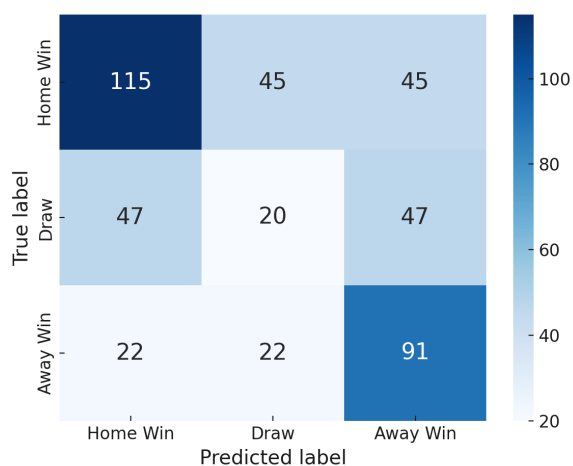


Figure A.22: World Cup Europe – Logistic Regression

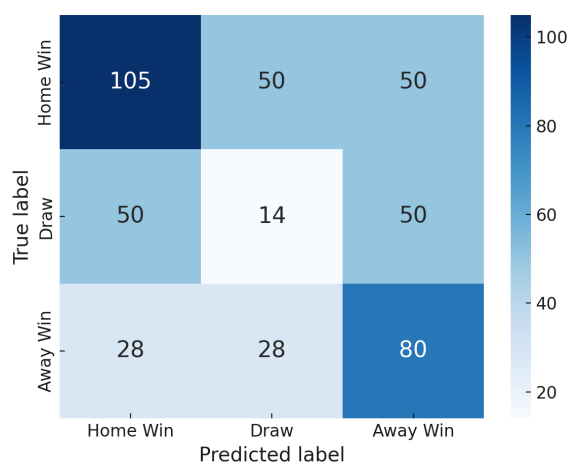


Figure A.23: World Cup Europe – XGBoost

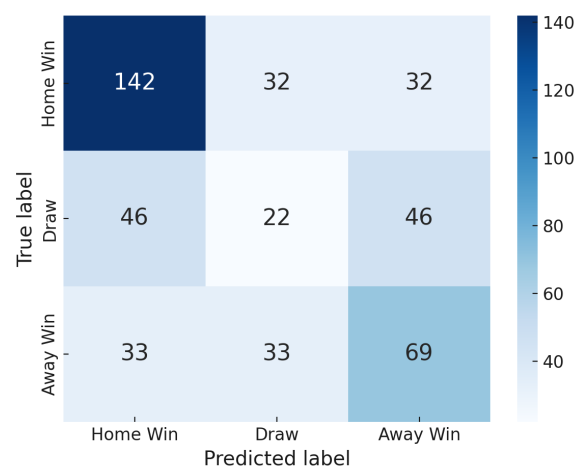


Figure A.24: World Cup Europe–Random Forest